June 15th, 2026

Hospital Fixed-Asset Tracking System Requirements and Use Case Specification Document

Imperial Healthtech

Supervisor(s) : Choirunnisa Hapsari
Kemal Falatehan

Author : Farros Ramzy

Table of Contents

List of Figures

List of Tables

1. Introduction

The Hospital Asset Tracker System is designed to help hospital staff register, monitor, and manage the movement of the hospital's fixed assets. Each asset's location and the nodes' status are shown live to the user via the monitoring app.

This document defines the system's features and how each hospital staff member, or the product user (an actor in this case), interacts with the system. In addition, this document explains the main workflows that support asset tracking in a hospital environment. The use cases in this document focus on the system's functional behavior from the perspectives of the user and external systems. The document mentions conditions that must be satisfied before each action, what happens during the normal flow of each process, possible alternative flows, and the expected result after each use case is completed.

2. System Overview

The Hospital Asset Tracker System consists of several connected components: the Registration App, the Monitor App, the backend API, the Database, the MQTT broker, and the ESP-32-based RFID sensor nodes. The Registration App is used to manage users, register nodes, register and deregister assets, and approve or reject asset movement requests. The Monitor App is used to observe asset status, node status, alerts, activity logs, and movement requests.

RFID sensor nodes are placed in hospital rooms or at the registration desk. The Registration Desk Node scans RFID tags during asset registration and deregistration, while the Checkpoint Nodes detect RFID-tagged assets in hospital rooms. The nodes send scan, detection, and heartbeat messages through MQTT. The backend processes these messages, updates the database, and broadcasts live updates to the web applications through WebSocket. This allows hospital staff to monitor asset movement and node availability without manually refreshing the application.

3. Actors

The system has both human and technical actors. The human actors interact with the system via the Registration App or the Monitor App, depending on their assigned role. In addition, the user can interact directly with the node sensors during setup or installation.

These actors include the initial System Administrator, Admin, Test User, Technician, Registration Staff, and Monitor Staff. Each of them has different permissions that control which pages and actions the user can access.

On the other hand, technical actors support the automated tracking workflow. The Registration Desk Node scans RFID tags during asset registration, while Checkpoint Nodes detect assets in hospital rooms and send heartbeat messages to indicate that they are online. The MQTT broker delivers node messages between the firmware and the backend system. The Backend System processes user requests, MQTT messages, database updates, movement workflows, alerts, and WebSocket live update broadcasts.

4. User Requirements

This section defines the user requirements of the Hospital Asset Tracker System. User requirements describe what the system must provide from the perspective of hospital staff, administrators, and operational users. These requirements cover authentication, role-based access, user management, node management, asset registration, asset movement, monitoring, alerts, MQTT communication, WebSocket updates, system settings, and first-admin setup.

Each user requirement is assigned a unique requirement ID so it can be traced to one or more use cases. This traceability helps confirm that the system behavior described in the use cases supports the needs of the intended users.

These user requirements are shown in the table below.

Table 4. 1 List of user requirements.

No.	ID	User Requirement Description
1.	UR-001	The system shall allow authorized users to log in, log out, and access only the pages allowed by their assigned role.
2.	UR-002	The system shall allow all authenticated users to change their own passwords securely.
3.	UR-003	The system shall allow the admin to create, update, disable, and manage user accounts and roles.
4.	UR-004	The system shall display a role-based Registration Dashboard with only the modules and summaries available to the logged-in user.
5.	UR-005	The system shall allow authorized users to open the Node Registration page and manage hospital tracking nodes.
6.	UR-006	The system shall allow authorized users to view records of discovered, assigned, online, offline, disabled, and unknown nodes.
7.	UR-007	The system shall allow authorized users to assign, edit, unassign, delete, and identify physical nodes by blink command.
8.	UR-008	The system shall allow authorized users to open the Asset Registration page and manage registered assets.
9.	UR-009	The system shall allow authorized users to register a new RFID-tagged hospital asset using a registration desk node.
10.	UR-010	The system shall allow authorized users to deregister an existing active asset when it is no longer tracked.
11.	UR-011	The system shall allow monitor-side users to request and cancel asset movement between rooms.
12.	UR-012	The system shall allow registration-side users to approve or reject pending asset movement requests.
13.	UR-013	The system shall complete an approved asset movement when the asset is detected at the approved destination checkpoint node.
14.	UR-014	The system shall allow monitoring users to view asset location, assigned room, movement status, node status, alerts, and activity logs.
15.	UR-015	The system shall detect and show warnings for unknown RFID tags, offline nodes, and unauthorized asset movement.
16.	UR-016	The system shall receive RFID scans, RFID detections, and heartbeat messages from sensor nodes through MQTT.
17.	UR-017	The system shall broadcast live updates to the Registration App and Monitor App without requiring manual refresh.
18.	UR-018	The system shall prevent unauthorized roles from accessing restricted pages or performing restricted actions.
19.	UR-019	The system shall allow authorized users to view and manage system or MQTT configuration settings.
20.	UR-020	The system shall allow an initial system administrator to create the first admin account when no admin account exists.

5. Functional Requirements

Functional requirements describe the specific functions that the system shall provide to satisfy the user requirements. These requirements explain the required behavior of the Registration App, Monitor App, backend API, MQTT communication, WebSocket updates, database updates, and role-based access control.

The functional requirements are written in a testable format for use during implementation, verification, and system testing. Each functional requirement can be traced to related user requirements and use cases in the traceability matrix.

The functional requirements of this system are listed in the table below.

Table 5. 1 List of system functional requirements.

No.	ID	Functional Requirement Description
1.	FR-001	The system shall provide a login function for authorized users in the Registration App and Monitor App.
2.	FR-002	The system shall verify the user account, password, active status, and role during login.
3.	FR-003	The system shall return the authenticated user's token and role after a successful login.
4.	FR-004	The system shall allow logged-in users to sign out and clear their active session data.
5.	FR-005	The system shall allow authenticated users to change their own password by entering the current password, new password, and confirmation password.
6.	FR-006	The system shall verify the current password before updating the stored password hash.
7.	FR-007	The system shall allow the admin to create, update, disable, and manage user accounts.
8.	FR-008	The system shall prevent unsafe admin account actions, such as disabling or demoting the last active admin account.
9.	FR-009	The system shall restrict protected pages and actions based on the authenticated user's assigned role.
10.	FR-010	The system shall display a Registration Dashboard with navigation links and summary cards based on the logged-in user's role.
11.	FR-011	The system shall hide node-related dashboard information when the user's role is not allowed to access node management.
12.	FR-012	The system shall hide asset-related dashboard information when the user's role is not allowed to access asset management.
13.	FR-013	The system shall allow authorized users to open the Node Registration page.
14.	FR-014	The system shall retrieve and display node records, including discovered, assigned, online, offline, disabled, and unknown nodes.
15.	FR-015	The system shall display node details, including device name, role, status, hospital, room, and the last ping time.
16.	FR-016	The system shall allow authorized users to assign an eligible node to a hospital, node role, alias, and room when required.
17.	FR-017	The system shall allow authorized users to edit the assignment data for an already-assigned node.
18.	FR-018	The system shall allow authorized users to send a blink command to an assigned online node for physical identification.
19.	FR-019	The system shall publish node blink commands through MQTT and process node acknowledgment messages when available.
20.	FR-020	The system shall allow authorized users to unassign a node after validating that the node can be safely unassigned.
21.	FR-021	The system shall allow authorized users to delete eligible node records after confirmation.
22.	FR-022	The system shall allow authorized users to open the Asset Registration page and view registered asset data.
23.	FR-023	The system shall display asset information, including asset name, RFID tag ID, status, last known location, and available actions.
24.	FR-024	After validating a registration-scan message, the system shall make the scanned RFID tag ID available to the Asset Registration workflow.
25.	FR-025	The system shall display the latest scanned RFID tag ID on the Asset Registration page.
26.	FR-026	The system shall allow authorized users to register a new asset using an RFID tag ID, asset name, and assigned room.
27.	FR-027	The system shall prevent registration of an RFID tag that is already registered as an active asset.
28.	FR-028	The system shall mark a newly registered asset as pending placement until it is detected in its assigned room.
29.	FR-029	The system shall allow authorized users to deregister an existing active asset.
30.	FR-030	The system shall update a deregistered asset, so it is no longer treated as an actively tracked hospital asset.
31.	FR-031	The system shall allow monitor-side users to request asset movement by selecting an active asset and a destination room.
32.	FR-032	The system shall reject duplicate movement requests when another pending request already exists for the selected asset.
33.	FR-033	The system shall allow monitor-side users to cancel a pending movement request.
34.	FR-034	The system shall prevent cancellation of movement requests that have already been approved, rejected, or completed.
35.	FR-035	The system shall display pending movement requests to authorized registration-side users.
36.	FR-036	The system shall allow authorized registration-side users to approve a pending movement request.
37.	FR-037	The system shall set the asset movement state to in transit after a movement request is approved.
38.	FR-038	The system shall allow authorized registration-side users to reject a pending movement request.
39.	FR-039	The system shall update the asset movement state after a movement request is rejected.
40.	FR-040	The system shall complete an approved movement when the asset is detected at the approved destination checkpoint node.
41.	FR-041	The system shall update the assigned room, current room, and flow status for the asset after a movement is completed.
42.	FR-042	The system shall display a Monitor Dashboard that includes assets, nodes, and a warning summary.
43.	FR-043	The system shall allow monitoring users to view each asset's current location, assigned room, status, and movement state.
44.	FR-044	The system shall allow monitoring users to view node availability and node status in the Monitor App.
45.	FR-045	The system shall allow monitoring users to view system alerts, including alert category, severity, message, and time.

46.	FR-046	The system shall allow monitoring users to view activity logs showing important assets, nodes, movement, user management, and system events.
47.	FR-047	The system shall create or display an unknown tag alert when an RFID tag is detected, but no active asset matches the tag ID.
48.	FR-048	The system shall detect offline nodes when heartbeat messages are not received within the expected time.
49.	FR-049	The system shall display an offline node alert when a node is considered unavailable.
50.	FR-050	The system shall detect unauthorized movement when an asset is detected outside its expected room without a valid approved movement request.
51.	FR-051	The system shall display an unauthorized movement or wrong-location warning to monitoring users.
52.	FR-052	The backend shall subscribe to and receive registration-scan messages from configured Registration Desk Node MQTT topics.
53.	FR-053	The system shall receive RFID detection messages from Checkpoint Nodes through MQTT.
54.	FR-054	The system shall receive heartbeat messages from Registration Desk Nodes and Checkpoint Nodes through MQTT.
55.	FR-055	The system shall use the MQTT broker to deliver scan, detection, heartbeat, and command messages between nodes and the backend.
56.	FR-056	The system shall validate incoming node messages before using them to update asset or node data.
57.	FR-057	The system shall open and maintain WebSocket connections for authenticated frontend clients.
58.	FR-058	The system shall broadcast asset, node, alert, activity, and movement updates to connected frontend clients through WebSocket.
59.	FR-059	The frontend shall update the visible tables, badges, dashboard counts, and alerts upon receiving a supported WebSocket event.
60.	FR-060	The frontend shall attempt to reconnect when the WebSocket connection is disconnected.
61.	FR-061	The frontend shall safely ignore unsupported or unknown WebSocket event types.
62.	FR-062	The backend shall reject restricted API requests when the authenticated user's role lacks the necessary permissions.
63.	FR-063	The frontend shall hide or block navigation options that are not allowed for the logged-in user's role.
64.	FR-064	The system shall allow authorized users to open the System Settings page.
65.	FR-065	The system shall retrieve current MQTT/system settings from the backend.
66.	FR-066	The system shall allow admin or authorized test users to update the MQTT/system settings.
67.	FR-067	The system shall validate setting values before saving them.
68.	FR-068	The system shall prevent unauthorized roles from changing system settings.
69.	FR-069	The system shall allow the first admin account to be created during setup when no admin account exists.

6. Non-Functional Requirements

The non-functional requirements describe the system's quality attributes and operational constraints, such as security, reliability, availability, performance, usability, maintainability, compatibility, data integrity, auditability, and scalability.

These requirements do not directly describe individual user actions. Instead, they define how well the system should perform and how safely, reliably, and maintainably the system should operate. They are important because the system handles hospital asset information, user roles, sensor communication, and live operational updates.

The list of non-functional requirements and their description can be found in the table below.

Table 6. 1 List of system non-functional requirements.

No.	ID	Category	Non-Functional Requirement Description
1.	NFR-001	Security	The system shall require users to log in before accessing protected pages in the Registration App or Monitor App.
2.	NFR-002	Security	The system shall restrict page access and actions based on the authenticated user's assigned role.
3.	NFR-003	Security	The system shall store user passwords securely using password hashing instead of storing plaintext passwords.
4.	NFR-004	Security	The system shall reject login attempts from disabled user accounts.
5.	NFR-005	Security	The system shall prevent unsafe admin account actions, such as disabling or demoting the last active admin account.
6.	NFR-006	Security	The system shall clear local authentication data when the user logs out or when the session is no longer valid.
7.	NFR-007	Security	The system should be deployed over HTTPS to ensure that authentication data and API communication are transmitted securely.

8.	NFR-008	Security	For production deployment, the system should avoid exposing sensitive configuration values, such as provisioning secrets, database credentials, and production tokens, in public source code.
9.	NFR-009	Reliability	The system shall continue to display the most recent stored asset and node data even when live WebSocket updates are temporarily unavailable.
10.	NFR-010	Reliability	The frontend shall attempt to reconnect when the WebSocket connection is interrupted.
11.	NFR-011	Reliability	The system shall detect offline nodes when expected heartbeat messages are not received within the configured timeout period.
12.	NFR-012	Reliability	The system shall validate incoming MQTT messages before using them to update asset or node data.
13.	NFR-013	Reliability	The system shall handle unknown RFID tags, invalid nodes, and unsupported WebSocket event types without crashing the application.
14.	NFR-014	Availability	The backend API should remain available for normal registration, monitoring, and movement workflows during expected operating hours.
15.	NFR-015	Availability	If the backend, database, MQTT broker, or WebSocket service is unavailable, the frontend shall show a clear error or connection status message to the user.
16.	NFR-016	Performance	The system should display page data within a reasonable loading time under normal network conditions.
17.	NFR-017	Performance	The system should update the frontend asset and node status shortly after the backend receives a valid MQTT or movement event.
18.	NFR-018	Performance	The system should support searching, sorting, filtering, and pagination without making asset and node tables difficult to use.
19.	NFR-019	Usability	The user interface shall clearly show the current status of assets, nodes, alerts, and movement requests.
20.	NFR-020	Usability	The system shall use clear visual indicators, such as badges, colors, and warning states, to help users identify important conditions of assets and nodes.
21.	NFR-021	Usability	The system shall provide clear success, error, loading, and empty-state messages for important user actions.
22.	NFR-022	Usability	The system shall hide or turn off actions that the current user role is not allowed to perform.
23.	NFR-023	Usability	The application layout should remain usable on common desktop, half-window, tablet, and mobile screen sizes.
24.	NFR-024	Usability	Important workflows, such as registering assets, assigning nodes, requesting movement, and approving movement, should be understandable without requiring technical knowledge of MQTT, WebSocket, or database behavior.
25.	NFR-025	Maintainability	The frontend, backend, and firmware code should be organized into small, readable modules to support future maintenance.
26.	NFR-026	Maintainability	Reusable logic should be placed in services, hooks, utilities, or helper modules rather than duplicated across pages.
27.	NFR-027	Maintainability	The system should use consistent naming for roles, asset statuses, node statuses, movement states, and event types.
28.	NFR-028	Maintainability	The system should include tests for critical backend workflows, such as authentication, role access, node management, asset registration, and asset movement.
29.	NFR-029	Compatibility	The web applications should run on modern desktop browsers such as Chrome, Edge, and Firefox.
30.	NFR-030	Compatibility	The sensor node firmware should support the selected ESP32-based hardware, RFID reader, Wi-Fi connection, and MQTT communication setup.
31.	NFR-031	Data Integrity	The system shall prevent duplicate active registration of the same RFID tag.
32.	NFR-032	Data Integrity	The system shall preserve consistency between the assigned asset room, the current room, the movement request state, and the flow status.
33.	NFR-033	Data Integrity	The system shall not complete an asset movement unless the asset is detected at the approved destination checkpoint node.
34.	NFR-034	Data Integrity	The system shall keep asset and node status records consistent after registration, deregistration, movement approval, movement rejection, heartbeat updates, and RFID detections.
35.	NFR-035	Auditability	The system should maintain activity records for important events, including node status changes, asset detections, movement requests, approvals, rejections, and warnings.
36.	NFR-036	Scalability	The system should be designed so that additional checkpoint nodes and tracked assets can be added without requiring major changes to the system architecture.

7. Use Case List

This section summarizes all use cases identified for the Hospital Asset Tracker System. The use cases are grouped by feature area, including account authentication and management, registration app access, node management, asset management, asset movement, monitoring, alerts and activity, real-time node communication, and system settings.

The use-case list provides a high-level overview of the system functions, the primary actors involved, and the feature group for each use case. This list helps readers understand the scope of the system before reading the detailed use-case descriptions.

The list below shows all the proper use cases of the Hospital Asset Tracker System.

Table 7. 1 List of use-cases with actors and feature groups.

Use Case ID	Use Case Name	Primary Actor(s)	Feature Group
UC-001	Create First Admin Account	❖ Initial System Admin	Account Authentication and Management
UC-002	Login	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff	Account Authentication and Management
UC-003	Logout	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff	Account Authentication and Management
UC-004	Change Own Password	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff	Account Authentication and Management
UC-005	Manage User Accounts	❖ Admin	Account Authentication and Management
UC-006	View Registration Dashboard	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff	Common Registration App Access
UC-007	Open Change Password Page	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff	Common Registration App Access
UC-008	Open User Accounts Page	❖ Admin	Common Registration App Access
UC-009	Open Node Registration Page	❖ Admin ❖ Test User ❖ Technician	Node And Asset Registration App Access
UC-010	Open Asset Registration Page	❖ Admin ❖ Test User ❖ Registration Staff	Node And Asset Registration App Access
UC-011	View Nodes	❖ Admin ❖ Test User ❖ Technician	Node Management
UC-012	Assign Node	❖ Admin ❖ Test User ❖ Technician	Node Management
UC-013	Edit Node Assignment	❖ Admin ❖ Test User ❖ Technician	Node Management

UC-014	Identify Node by Blink	❖ Admin ❖ Test User ❖ Technician	Node Management
UC-015	Unassign Node	❖ Admin ❖ Test User ❖ Technician	Node Management
UC-016	Delete Node	❖ Admin ❖ Test User ❖ Technician	Node Management
UC-017	View Assets	❖ Admin ❖ Test User ❖ Registration Staff	Asset Management
UC-018	Register Asset	❖ Admin ❖ Test User ❖ Registration Staff	Asset Management
UC-019	Deregister Asset	❖ Admin ❖ Test User ❖ Registration Staff	Asset Management
UC-020	Scan the RFID Tag at the Registration Desk	❖ Registration Desk Node	Asset Management
UC-021	Request Asset Movement	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Movement Request
UC-022	Cancel Movement Request	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Movement Request
UC-023	Approve Movement Request	❖ Admin ❖ Test User ❖ Registration Staff	Asset Movement Management
UC-024	Reject Movement Request	❖ Admin ❖ Test User ❖ Registration Staff	Asset Movement Management
UC-025	Complete Approved Movement	❖ Checkpoint Node ❖ Backend System ❖ Observing Actors (Admin/Staff)	Asset Movement Management
UC-026	View Monitor Dashboard	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Tracker Monitor
UC-027	View Asset Monitor	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Tracker Monitor
UC-028	View Node Monitor	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Tracker Monitor
UC-029	View Current Asset Status	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Tracker Monitor
UC-030	Receive Live Updates	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Tracker Monitor
UC-031	View Alerts	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Alert and Activity
UC-032	View Activity Logs	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Alert and Activity
UC-033	See Unknown Tag Alert	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Alert and Activity
UC-034	See Offline Node Alert	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Alert and Activity

UC-035	See Unauthorized Movement Warning	❖ Admin ❖ Test User ❖ Monitor Staff	Asset Alert and Activity
UC-036	Send Registration Scan	❖ Registration Node	Node Realtime Communication
UC-037	Send RFID Detection	❖ Checkpoint Node	Node Realtime Communication
UC-038	Send Heartbeat	❖ Registration Desk Node ❖ Checkpoint Node	Node Realtime Communication
UC-039	Deliver MQTT Messages	❖ MQTT Broker	Node Realtime Communication
UC-040	Broadcast WebSocket Update	❖ Backend System	Node Realtime Communication
UC-041	Manage System Settings	❖ Admin ❖ Test User	System Settings

8. Use Case Descriptions

This section describes each use case in detail. Each use-case description includes the use case ID, name, actor, description, precondition, basic flow, alternative flow, and post-condition. These details explain how users or external system actors interact with the system and what result is expected after each interaction.

The use-case descriptions support system design, implementation, and testing. They help ensure that system workflows are clear, consistent, and traceable to user and functional requirements.

8.1. Account Authentication and Management

The use cases in the section below relate to system access and account management. These use cases include creating the first admin account, logging in, logging out, changing a password, and managing user accounts. They form the system's security foundation because all protected pages and actions depend on authentication and role-based access control.

The first-admin setup use case is only available when no admin account exists. After the first admin account is created, normal account management is handled by the Admin role through the User Accounts page.

Details can be found in this diagram and the use case tables below it.

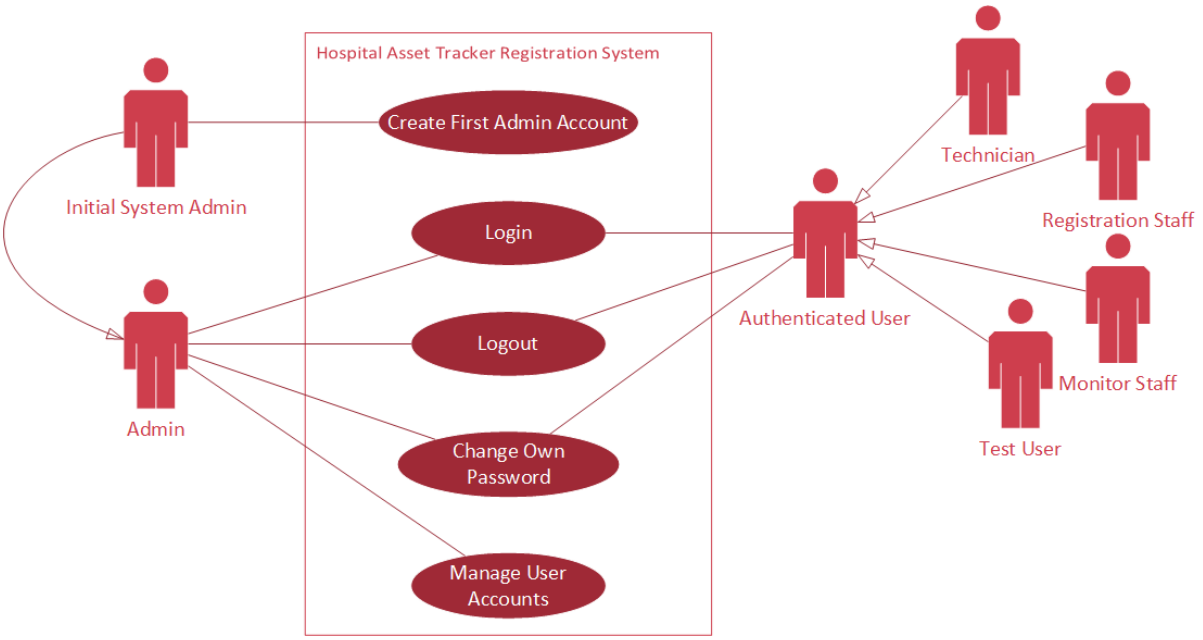

Figure 8. 1 Authentication and account management use case diagram.

8.1.1. UC-001 – Create First Admin Account

Table 8. 1. 1 Data of use case UC-001.

Use Case ID:	UC-001
Name:	Create First Admin Account
Actor:	❖ Initial System Administrator
Description:	Allows the initial system administrator to create the first admin account when the system has no existing admin account. This setup process is only available during initial system setup and should no longer be accessible after the first admin account has been created.
Pre-Condition:	The system is running, the backend authentication service is available, and no admin account currently exists.
Basic Flows:	1. The initial system administrator opens the first-admin setup page. 2. The frontend requests the admin setup status from the backend. 3. The backend checks whether an admin account already exists. 4. If no admin account exists, the backend allows the setup page to continue. 5. The initial system administrator enters the required admin account information, such as name, username, or email, and password. 6. The frontend validates the required fields and password confirmation. 7. The frontend sends the setup request to the backend. 8. The backend validates that no admin account exists before creating the account. 9. The backend securely hashes the password and creates the first admin account. 10. The system disables or blocks the first-admin setup process after the account is created. 11. The frontend shows a success message and redirects the user to the login page.
Alternative Flows:	1. If an admin account already exists, the system blocks the setup page and redirects the user to the login page. 2. If required fields are missing, the frontend or backend shows a validation error. 3. If the password and confirmation password do not match, the system rejects the form. 4. If the username or email is invalid, the system shows an error message. 5. If the backend cannot create the account, the setup page shows an error, and the first admin account is not created.
Post-Condition:	A first admin account is created successfully. The first-admin setup process is no longer available, and the new admin can log in and manage the system through normal admin functions.

8.1.2. UC-002 – Login

Table 8. 1. 2 Data of use case UC-002.

Use Case ID:	UC-002
Name:	Login
Actor:	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff
Description:	Allows an authorized user to access the system using registered account credentials.
Pre-Condition:	The user has an active account, and the backend authentication service is available.
Basic Flows:	1. The user opens the Registration App or Monitor App login page. 2. The user enters their email or username and password. 3. The frontend sends the credentials to the backend authentication API. 4. The backend verifies the account, password hash, active status, and role. 5. The backend returns a JWT token and user role. 6. The frontend stores the session and redirects the user to an allowed page.
Alternative Flows:	1. If the credentials are invalid, the system shows a login error. 2. If the account is disabled, the system rejects the login. 3. If the user role is not allowed for the app, the system blocks access.
Post-Condition:	The user is logged in and can access pages allowed by the assigned role.

8.1.3. UC-003 – Logout

Table 8. 1. 3 Data of use case UC-003.

Use Case ID:	UC-003
Name:	Logout
Actor:	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff
Description:	Allows a logged-in user to securely end the active session.
Pre-Condition:	The user is currently logged in.
Basic Flows:	1. The user clicks the Sign out button. 2. The frontend clears stored authentication data. 3. The frontend closes or resets the protected application state. 4. The user is redirected to the login page.
Alternative Flows:	1. If the session has already expired, the front end still clears local session data and shows the login page.
Post-Condition:	The user is logged out and cannot access protected pages without logging in again.

8.1.4. UC-004 – Change Password

Table 8. 1. 4 Data of use case UC-004.

Use Case ID:	UC-004
Name:	Change Own Password
Actor:	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff
Description:	Allows a logged-in user to change their own account password.
Pre-Condition:	The user is logged in and knows the current password.
Basic Flows:	1. The user opens the Change Password page. 2. The user enters the current password, new password, and confirmation password. 3. The frontend validates that the new password and confirmation match. 4. The backend verifies the current password. 5. The backend updates the stored password hash. 6. The system shows a success message.
Alternative Flows:	1. If the current password is wrong, the system rejects the request. 2. If the new password is too weak or does not match the confirmation, the system shows a validation error.
Post-Condition:	The account password is updated and must be used for future logins.

8.1.5. UC-005 – Manage User Accounts

Table 8. 1. 5 Data of use case UC-005.

Use Case ID:	UC-005
Name:	Manage User Accounts
Actor:	❖ Admin
Description:	Allows the admin to create, update, disable, or manage user accounts and roles.

Pre-Condition:	The user is logged in as Admin.
Basic Flows:	1. The admin opens the User Accounts page. 2. The system loads the existing user list. 3. The admin creates a new user or updates an existing user. 4. The frontend sends the account change to the backend. 5. The backend validates admin permission and account safety rules. 6. The backend saves the user account update. 7. The frontend refreshes the user list and shows the result.
Alternative Flows:	1. If the admin tries to disable or demote the last active admin, the system rejects the action. 2. If the selected role is invalid, the system shows an error. 3. If the username or email already exists, the system rejects the creation.
Post-Condition:	The user account data is created or updated according to admin permission rules.

8.2. Common Registration App Access

This section describes the common access use cases in the Registration App. These use cases include viewing the Registration Dashboard, opening the Change Password page, and opening the User Accounts page. Access to these pages depends on the logged-in user's role.

The Registration Dashboard acts as the main entry point for registration-side users. It displays role-based navigation and summary information, so each user only sees the modules and actions relevant to their permission level.

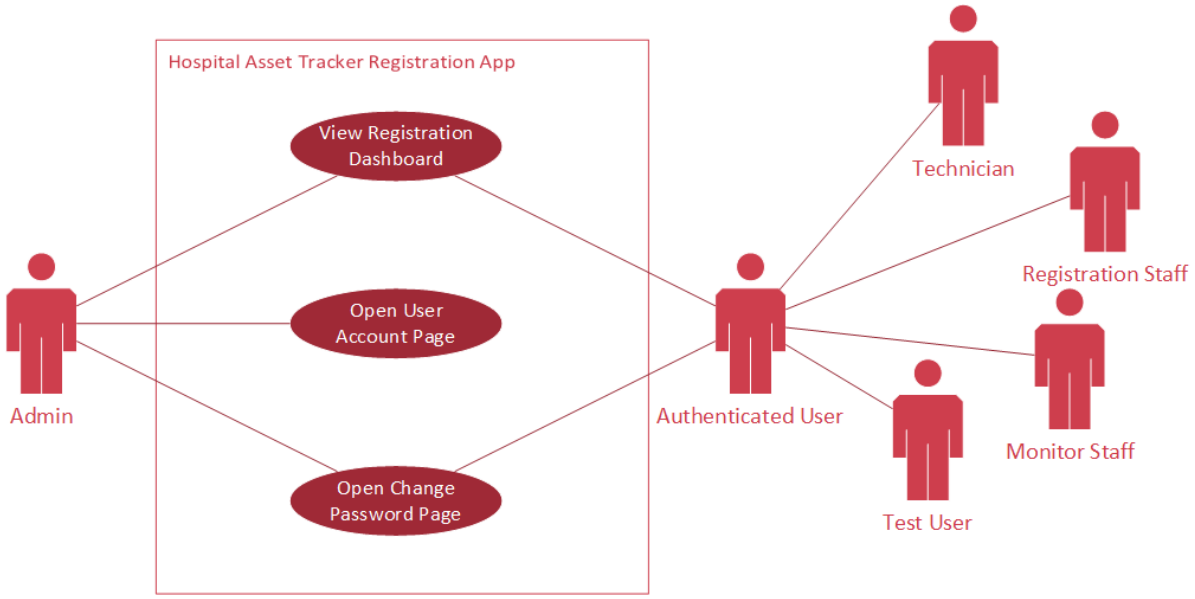

Figure 8. 2 User main access per-role use case diagram.

8.2.1. UC-006 – View Registration Dashboard

Table 8. 2. 1 Data of use case UC-006.

Use Case ID:	UC-006
Name:	View Registration Dashboard
Actor:	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff

Description:	Displays the registration-side dashboard with cards and links tailored to the user's role.
Pre-Condition:	The user is logged in to the Registration App.
Basic Flows:	1. The user opens the Registration Dashboard. 2. The frontend reads the logged-in user role. 3. The dashboard loads the node and/or asset data allowed for that role. 4. The dashboard calculates summary cards and access cards. 5. The system displays only the pages and information that the role is allowed to access
Alternative Flows:	1. If the role cannot access nodes, node overview cards are hidden. 2. If the role cannot access assets, asset overview cards are hidden. 3. If data loading fails, the page shows an error or fallback state.
Post-Condition:	The user sees a role-appropriate dashboard and navigation options.

8.2.2. UC-007 – Open Change Password Page

Table 8. 2. 2 Data of use case UC-007.

Use Case ID:	UC-007
Name:	Open Change Password Page
Actor:	❖ Admin ❖ Test User ❖ Technician ❖ Registration Staff ❖ Monitor Staff
Description:	Allows any authenticated user to access the account security page.
Pre-Condition:	The user is logged in.
Basic Flows:	1. The user opens the Change Password page. 2. The frontend checks that the user has an active session. 3. The page displays current password, new password, and confirmation fields. 4. The user can submit a password change request.
Alternative Flows:	1. If the session has expired, the user is redirected to the login page.
Post-Condition:	The password change form is available to the authenticated user.

8.2.3. UC-008 – Open User Accounts Page

Table 8. 2. 3 Data of use case UC-008.

Use Case ID:	UC-008
Name:	Open User Accounts Page
Actor:	❖ Admin
Description:	Allows the admin to access the account management page.
Pre-Condition:	The user is logged in as Admin.
Basic Flows:	1. The admin selects the User Accounts page. 2. The frontend checks admin-only access. 3. The frontend requests the user list from the backend. 4. The backend validates admin permission and returns account data. 5. The page displays account creation and management tools.

Alternative Flows:	1. If a non-admin tries to open the page, the frontend blocks the route, or the backend returns access denied.
Post-Condition:	The admin can manage user accounts.

8.3. Node and Asset Registration Access

The Node Registration page and Asset Registration page are separate because different user roles handle node management and asset registration, and because they support different operational workflows.

The Node Registration page is mainly used by Admin, Test User, and Technician roles to manage sensor nodes. The Asset Registration page is primarily used by the Admin, Test User, and Registration Staff roles to register and deregister assets and handle movement approval workflows.

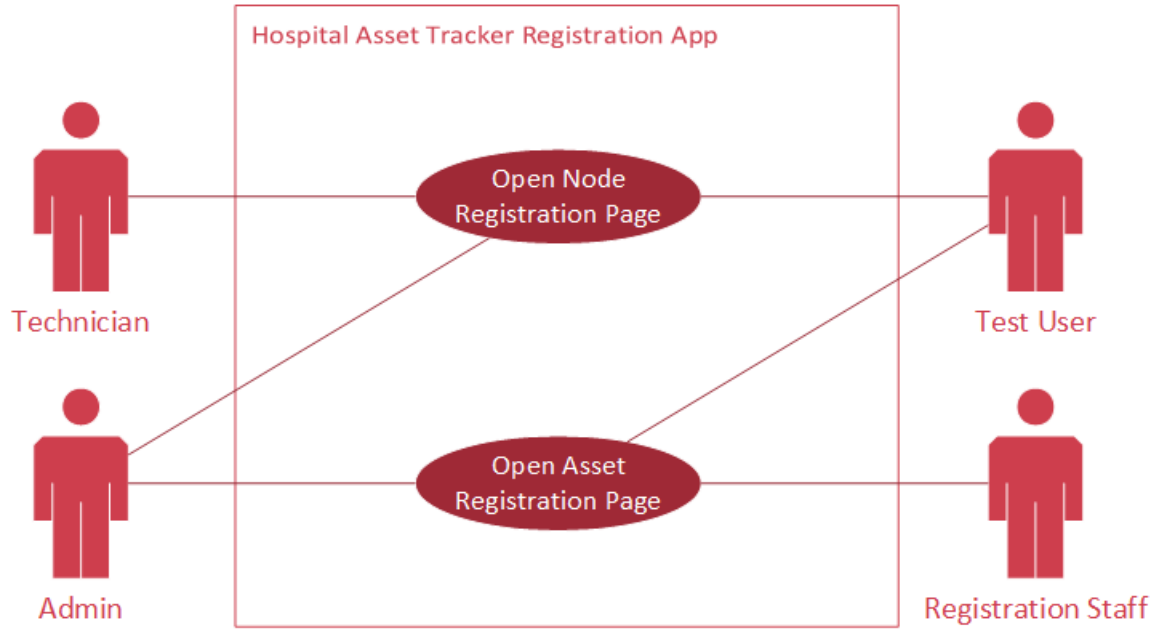

Figure 8. 3 Node and asset management access use case diagram.

8.3.1. UC-009 – Open Node Registration Page

Table 8. 3. 1 Data of use case UC-009.

Use Case ID:	UC-009
Name:	Open Node Registration Page
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Allows authorized registration-side users to access node registration and node management.
Pre-Condition:	The user is logged in with a role that allows them to manage nodes.

Basic Flows:	1. The user selects the Node Registration page. 2. The frontend checks the user role. 3. The page requests the node list from the backend. 4. The backend validates permission and returns node data. 5. The frontend displays node status, role, hospital, room, and actions.
Alternative Flows:	1. If the role is not allowed, the user is redirected or shown access denied. 2. If the backend cannot load nodes, the page shows an error message.
Post-Condition:	The user can view and manage nodes based on their role permissions.

8.3.2. UC-010 – Open Asset Registration Page

Table 8. 3. 2 Data of use case UC-010.

Use Case ID:	UC-010
Name:	Open Asset Registration Page
Actor:	❖ Admin ❖ Test User ❖ Registration Staff
Description:	Allows authorized users to access asset registration, deregistration, and movement approval features.
Pre-Condition:	The user is logged in with a role that allows them to manage registration-side assets.
Basic Flows:	1. The user opens the Asset Registration page. 2. The frontend verifies the user role. 3. The page loads registered assets, registration nodes, and pending movement requests. 4. The frontend displays registration, deregistration, and movement decision sections.
Alternative Flows:	1. If the user role is not allowed, access is denied. 2. If there are no online registration nodes, registration actions are limited.
Post-Condition:	The user can use the asset registration functions allowed by the role.

8.4. Node Management

This section describes the use cases for managing RFID sensor nodes. Sensor nodes are important because they allow the system to detect asset locations, send heartbeat messages, and identify physical devices through blink commands.

Node management includes viewing nodes, assigning nodes, editing node assignments, identifying nodes by blink, unassigning nodes, and deleting eligible node records. These actions ensure that each node is correctly connected to a hospital role, location, and tracking function.

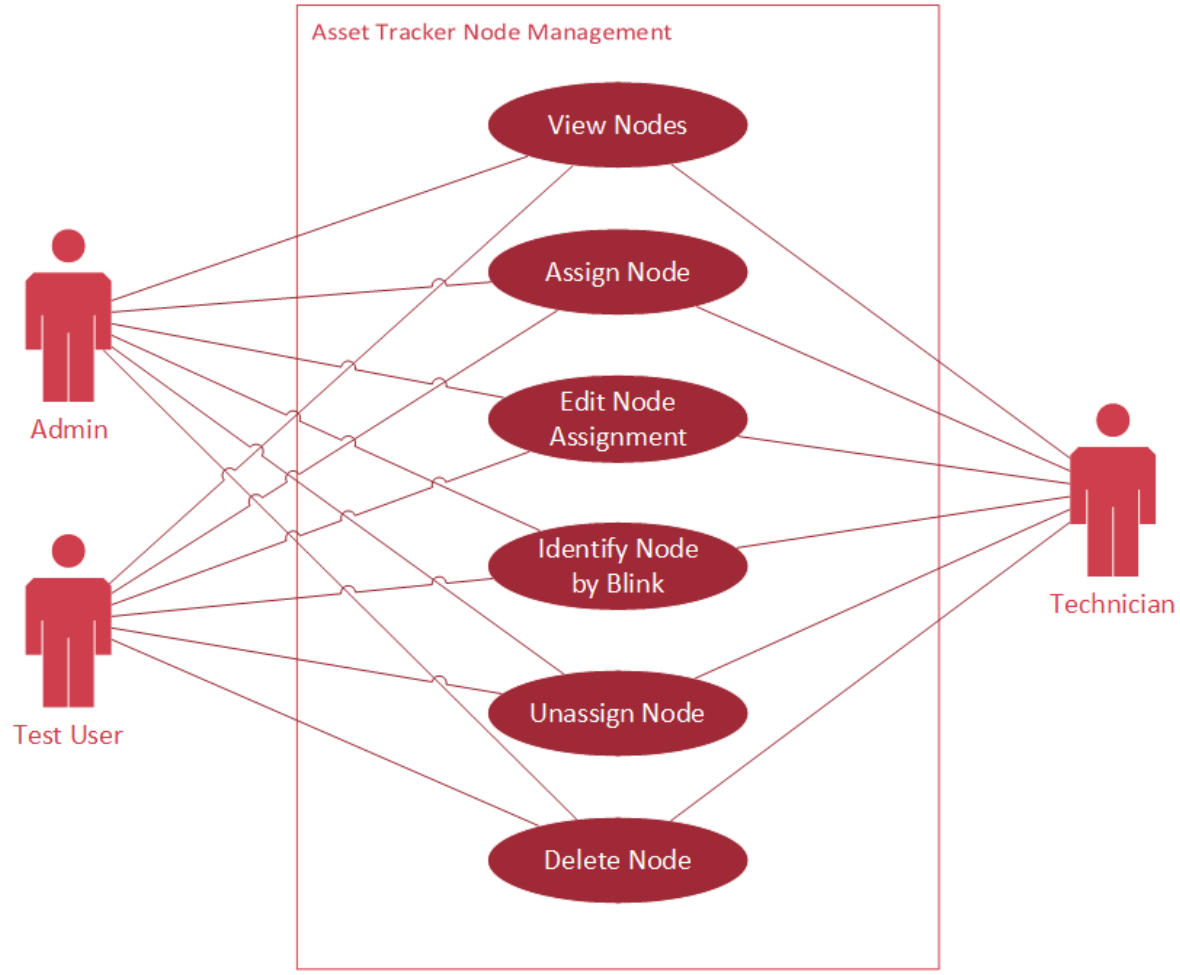

Figure 8. 4 Node management access use case diagram.

8.4.1. UC-011 – View Nodes

Table 8. 4. 1 Data of use case UC-011.

Use Case ID:	UC-011
Name:	View Nodes
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Allows authorized users to view discovered, assigned, online, offline, disabled, and unknown node records.

Precondition:	The user has node management permission.
Basic Flows:	1. The user opens the Node Registration page. 2. The frontend requests the node list. 3. The backend returns node records and latest status values. 4. The frontend formats node role, status, hospital, location, and last ping. 5. The user reviews the node table.
Alternative Flows:	1. If no nodes exist, the table shows an empty state. 2. If the backend is unavailable, the page shows a loading error.
Post-Condition:	The current node list is visible to the user.

8.4.2. UC-012 – Assign Node

Table 8. 4. 2 Data of use case UC-012.

Use Case ID:	UC-012
Name:	Assign Node
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Allows an authorized user to assign a discovered node to a hospital, role, and room.
Pre-Condition:	A discovered or eligible node exists, and the user has node management permission.
Basic Flows:	1. The user selects Assign on a discovered node. 2. The frontend displays the assignment form. 3. The user enters the alias, hospital, node role, and room when required. 4. The frontend sends the assignment request to the backend. 5. The backend validates the node and assignment data. 6. The backend stores the assignment and returns the updated node. 7. The frontend updates the node table.
Alternative Flows:	1. If the node role is Registration Desk, the room can be omitted or disabled based on the rule. 2. If required fields are missing, the system shows validation errors. 3. If the node is not eligible, the backend rejects the request.
Post-Condition:	The node is assigned and can be used in accordance with its role.

8.4.3. UC-013 – Edit Node Assignment

Table 8. 4. 3 Data of use case UC-013.

Use Case ID:	UC-013
Name:	Edit Node Assignment
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Allows an authorized user to update an already assigned node.
Pre-Condition:	The selected node is assigned, and the user has permission to manage nodes.
Basic Flows:	1. The user clicks Edit on an assigned node. 2. The frontend loads the current node assignment data into the form. 3. The user changes the alias, hospital, role, or room. 4. The frontend submits the update to the backend. 5. The backend validates and saves the changes. 6. The frontend refreshes or updates the node row.
Alternative Flows:	1. If the user changes a node to Registration Desk, the room field may be cleared. 2. If the update conflicts with validation rules, the backend rejects it.
Post-Condition:	The node assignment is updated in the database and shown in the UI.

8.4.4. UC-014 – Identify Node by Blink

Table 8. 4. 4 Data of use case UC-014.

Use Case ID:	UC-014
Name:	Identify Node by Blink
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Sends a blink command to a physical node so staff can identify it.
Pre-Condition:	The node is assigned and online, and MQTT communication is available.
Basic Flows:	1. The user clicks the Identify or Blink action for a node. 2. The frontend sends a command request to the backend. 3. The backend publishes a blink command through MQTT. 4. The node receives the command and blinks its LEDs. 5. The node sends an acknowledgment message. 6. The system shows the command status to the user.
Alternative Flows:	1. If the node is offline, the command is disabled or rejected. 2. If no acknowledgment is received, the system may show a timeout or pending status.
Post-Condition:	The selected physical node has been visually identified, or the user is informed that the command failed.

8.4.5. UC-015 – Unassign Node

Table 8. 4. 5 Data of use case UC-015.

Use Case ID:	UC-015
Name:	Unassign Node
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Removes the current hospital, role, and location assignment from a node without deleting it.
Pre-Condition:	The node is currently assigned, and the user has permission to manage nodes.
Basic Flows:	1. The user opens the node edit form. 2. The user chooses Unassign. 3. The frontend asks for confirmation when required. 4. The backend validates the node state. 5. The backend clears assignment fields or changes status according to the rule. 6. The frontend updates the table.
Alternative Flows:	1. If active assets or movements need the node, the system may reject the unassign request. 2. If the user cancels confirmation, no changes are made.
Post-Condition:	The node is no longer assigned to an active hospital location.

8.4.6. UC-016 – Delete Node

Table 8. 4. 6 Data of use case UC-016.

Use Case ID:	UC-016
Name:	Delete Node
Actor:	❖ Admin ❖ Test User ❖ Technician
Description:	Deletes an eligible node record from the system.
Pre-Condition:	The node is eligible for deletion, and the user has permission to manage nodes.

Basic Flows:	1. The user selects a node that can be deleted. 2. The user chooses Delete and confirms the action. 3. The frontend sends the delete request to the backend. 4. The backend validates that the node can be removed. 5. The backend deletes the node record. 6. The frontend removes the node from the table.
Alternative Flows:	1. If the node is active or assigned in a way that prevents deletion, the backend rejects the request. 2. If the user cancels confirmation, no deletion occurs.
Post-Condition:	The node record is removed from the system.

8.5. Asset Management

Asset management includes viewing registered assets, registering new assets, deregistering inactive assets, and receiving RFID scans from the Registration Desk Node.

Asset registration links an RFID tag to a hospital asset and assigns it to an expected room. After registration, the system can monitor whether the asset reaches the assigned room or appears in an unexpected location.

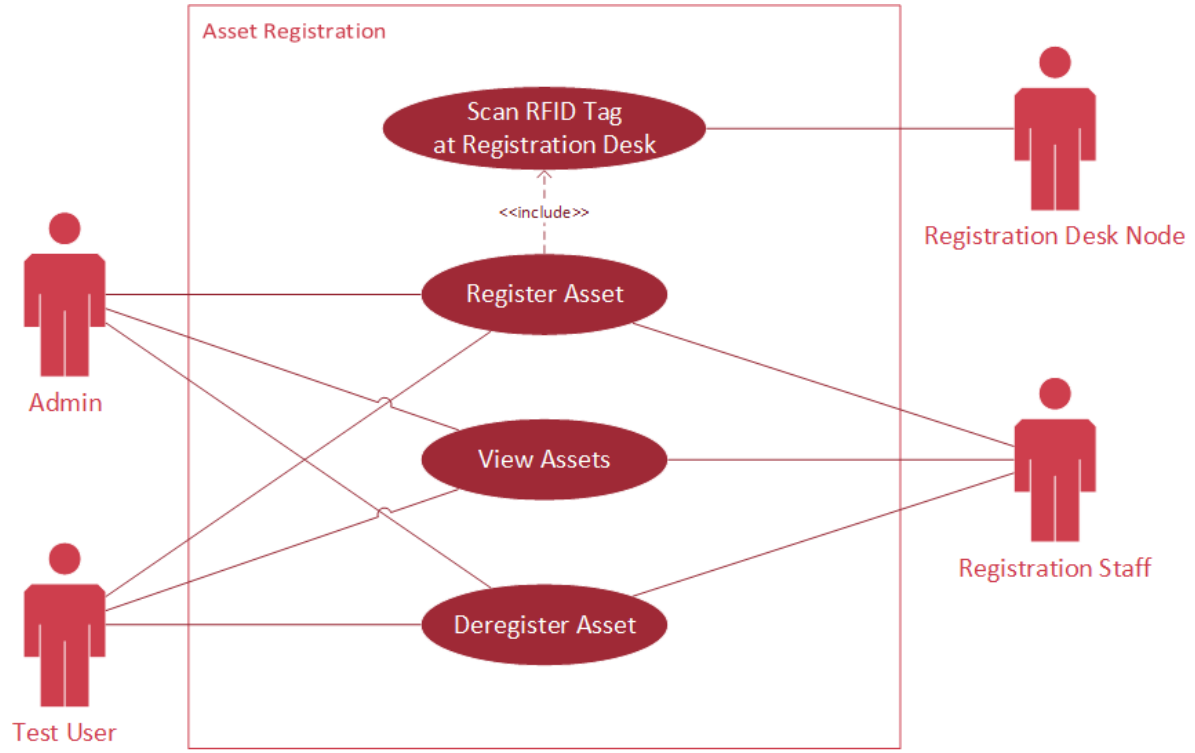

Figure 8. 5 Asset management access use case diagram.

8.5.1. UC-017 – View Assets

Table 8. 5. 1 Data of use case UC-017.

Use Case ID:	UC-017
Name:	View Assets

Actor:	❖ Admin ❖ Test User ❖ Registration Staff
Description:	Allows authorized users to view registered assets in the Registration App.
Pre-Condition:	The user has asset registration permission.
Basic Flows:	1. The user opens the Asset Registration page. 2. The frontend requests the asset list. 3. The backend validates permission and returns asset data. 4. The frontend displays the asset name, tag ID, status, last location, and actions. 5. The user searches, sorts, or reviews asset rows.
Alternative Flows:	1. If no assets exist, the page shows an empty state. 2. If loading fails, the page shows an error.
Post-Condition:	The user can view current registration-side asset data.

8.5.2. UC-018 – Register Asset

Table 8. 5. 2 Data of use case UC-018.

Use Case ID:	UC-018
Name:	Register Asset
Actor:	❖ Admin ❖ Test User ❖ Registration Staff
Description:	Allows an authorized user to register a new RFID-tagged hospital asset.
Pre-Condition:	The user has asset registration permission, a registration desk node is online, and an online Registration Desk Node has scanned a valid RFID tag.
Basic Flows:	1. The user opens the register asset form. 2. The system displays the scanned RFID tag ID. 3. The user enters the asset name and assigned room. 4. The user submits the registration form. 5. The backend checks that the tag is not already active. 6. The backend creates the asset record and marks it as pending placement. 7. The frontend displays the newly registered asset.
Alternative Flows:	1. If the tag is already registered, the system shows an error. 2. If no registration node is online, registration is blocked. 3. If required fields are missing, the frontend or backend rejects the form.
Post-Condition:	The asset is registered and is awaiting detection in its assigned room.

8.5.3. UC-019 – Deregister Asset

Table 8. 5. 3 Data of use case UC-019.

Use Case ID:	UC-019
Name:	Deregister Asset
Actor:	❖ Admin ❖ Test User ❖ Registration Staff
Description:	Allows an authorized user to deactivate or deregister an existing asset.
Pre-Condition:	The asset exists and is active, and the user has permission to register assets.

Basic Flows:	1. The user selects the asset to deregister. 2. The system displays deregistration details. 3. The user confirms the deregistration action. 4. The frontend sends the request to the backend. 5. The backend validates the asset and user permission. 6. The backend updates the asset status to deregistered or inactive. 7. The frontend updates the asset table.
Alternative Flows:	1. If the asset is already deregistered, the backend rejects the request. 2. If the user cancels confirmation, no change is made.
Post-Condition:	The asset is no longer treated as an actively tracked hospital asset.

8.5.4. UC-020 – Scan RFID Tag at Registration Desk

Table 8. 5. 4 Data of use case UC-020.

Use Case ID:	UC-020
Name:	Scan RFID Tag at the Registration Desk
Actor:	❖ Registration Desk Node
Description:	Allows the registration desk node to send a scanned RFID tag ID to the backend and frontend.
Pre-Condition:	The registration desk node is assigned, online, and connected to MQTT.
Basic Flows:	1. The RFID tag is scanned at the registration desk. 2. The registration desk node reads the tag ID. 3. The node publishes the scan message to MQTT. 4. The backend receives and validates the scan message. 5. The backend broadcasts the latest scanned tag to the Registration App. 6. The Asset Registration page displays the tag ID.
Alternative Flows:	1. If the node is not recognized, the backend ignores or warns about the scan. 2. If MQTT is disconnected, the scan does not reach the backend.
Post-Condition:	The scanned tag ID is available for asset registration or deregistration.

8.6. Asset Movement Request

This section describes how monitor-side users request and cancel asset movement. Movement requests are used to control and track the relocation of assets between hospital rooms. This prevents assets from being moved without approval and helps maintain an accurate expected location for each asset.

A movement request is created from the Monitor App and remains pending until it is approved or rejected by an authorized registration-side user. If the request is no longer needed, the requester can cancel it while it is still pending.

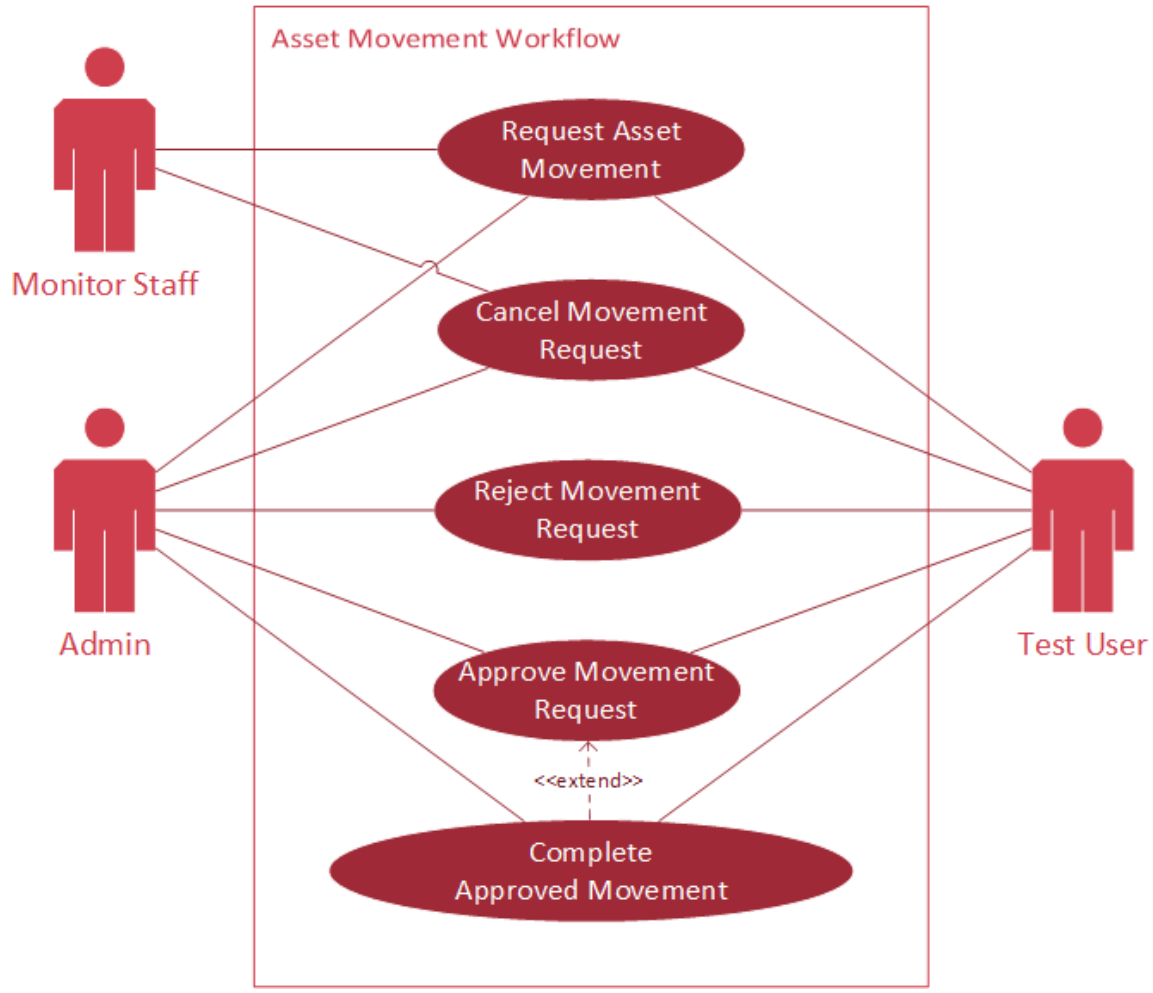

Figure 8. 6 Asset movement request and approval use case diagram.

8.6.1. UC-021 – Request Asset Movement

Table 8. 6. 1 Data of use case UC-021.

Use Case ID:	UC-021
Name:	Request Asset Movement
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows a monitoring-side user to request permission to move an asset to another room.
Pre-Condition:	The user has permission to monitor App movement, and the selected asset is active and eligible for movement.
Basic Flows:	1. The user opens the Monitor App asset table. 2. The user selects an asset and chooses Request Movement. 3. The user selects the destination room. 4. The frontend sends the movement request to the backend. 5. The backend validates the asset and movement state. 6. The backend creates a pending movement request. 7. The backend broadcasts the request to the Registration App and Monitor App.
Alternative Flows:	1. If another pending request already exists, the system rejects duplicate submission. 2. If the asset is not active, movement cannot be requested. 3. If the destination is invalid, the system shows an error.
Post-Condition:	A pending movement request is visible and awaits registration-side approval.

8.6.2. UC-022 – Cancel Movement Request

Table 8. 6. 2 Data of use case UC-022.

Use Case ID:	UC-022
Name:	Cancel Movement Request
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows the requester to cancel a pending movement request.
Pre-Condition:	A pending movement request exists for the selected asset.
Basic Flows:	1. The user selects an asset with a pending movement request. 2. The user clicks Cancel Request. 3. The frontend sends a cancel request to the backend. 4. The backend verifies that the request is still pending. 5. The backend cancels the request and updates the asset flow state. 6. The backend broadcasts the update to connected apps.
Alternative Flows:	1. If the request was already approved or rejected, cancellation is not allowed. 2. If the request no longer exists, the system shows an error.
Post-Condition:	The movement request has been canceled, and the asset is no longer awaiting approval for that request.

8.7. Asset Movement Management

This section describes how pending movement requests are approved, rejected, and completed. Registration-side users review movement requests and decide whether the asset may be moved to the requested destination.

When a movement request is approved, the asset is marked as in transit. The movement is only completed when the asset is detected at the approved destination checkpoint node. If the asset is detected in the wrong room or without an approved request, the system can display a warning instead of completing the movement.

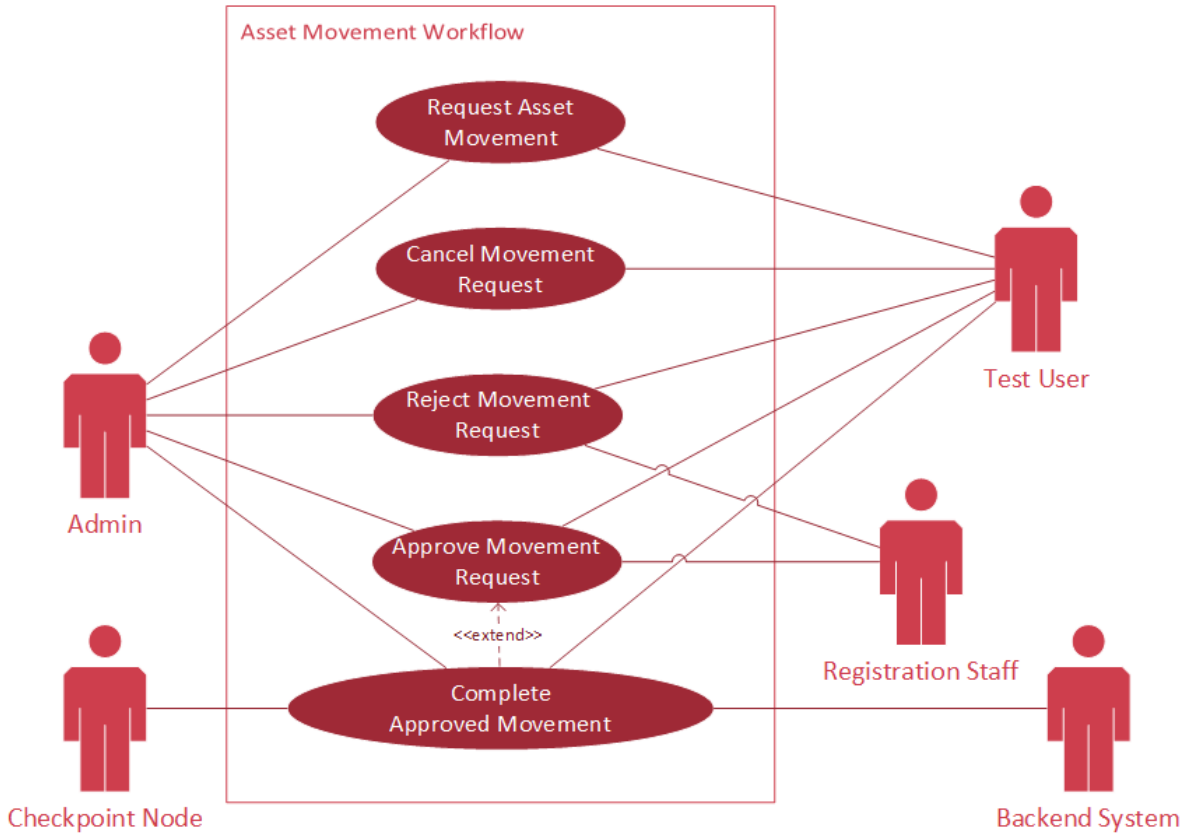

Figure 8. 7 Asset movement management use case diagram.

8.7.1. UC-023 – Approve Movement Request

Table 8. 7. 1 Data of use case UC-023.

Use Case ID:	UC-023
Name:	Approve Movement Request
Actor:	❖ Admin ❖ Test User ❖ Registration Staff
Description:	Allows an authorized registration-side user to approve a pending movement request.
Pre-Condition:	A pending movement request exists, and an authorized user is logged in to the Registration App.

Basic Flows:	1. The user opens the pending movement request list. 2. The user reviews the requested asset and destination. 3. The user selects Approve. 4. The frontend sends the approval decision to the backend. 5. The backend validates the movement request and registration node. 6. The backend marks the request as approved and sets the asset as in transit. 7. The backend broadcasts the approved movement status.
Alternative Flows:	1. If the request has already been handled, approval is rejected. 2. If required validation fails, the backend returns an error.
Post-Condition:	The asset is approved for movement and is expected to be detected at the destination room.

8.7.2. UC-024 – Reject Movement Request

Table 8. 7. 2 Data of use case UC-024.

Use Case ID:	UC-024
Name:	Reject Movement Request
Actor:	❖ Admin ❖ Test User ❖ Registration Staff
Description:	Allows an authorized registration-side user to reject a pending movement request.
Pre-Condition:	A movement request is pending, and the user has permission to make movement decisions.
Basic Flows:	1. The user opens the pending movement request list. 2. The user reviews the request details. 3. The user selects Reject. 4. The frontend sends the rejection decision to the backend. 5. The backend marks the request as rejected. 6. The backend updates the asset flow state. 7. The backend broadcasts the rejection result.
Alternative Flows:	1. If the request has already been approved or rejected, the system prevents another decision. 2. If the backend update fails, the page shows an error.
Post-Condition:	The request is marked as rejected. The asset's assigned room remains unchanged, and its movement-related display state returns to the appropriate non-pending state.

8.7.3. UC-025 – Complete Approved Movement

Table 8. 7. 3 Data of use case UC-025.

Use Case ID:	UC-025
Name:	Complete Approved Movement
Actor:	❖ Checkpoint Node ❖ Backend System ❖ Admin/Staff Users who can observe the result
Description:	Completes an approved movement when the asset is detected at the approved destination checkpoint node.
Pre-Condition:	An approved movement request is in place, and the asset is in transit.
Basic Flows:	1. The checkpoint node at the destination detects the asset RFID tag. 2. The node sends the detection through MQTT. 3. The backend loads the active movement request. 4. The backend confirms that the detected room matches the approved destination. 5. The backend completes the movement request. 6. The backend updates the asset assigned room, current room, and flow status. 7. The backend broadcasts movement completed and asset updated events.
Alternative Flows:	1. If the asset is detected in the wrong room, the system shows a warning instead of completing the movement. 2. If no approved movement exists, the detection may become unauthorized movement.
Post-Condition:	The movement request has been completed, and the asset is marked as available or in place at the approved destination.

8.8. Asset Tracker Monitor

This section describes the monitoring use cases available in the Monitor App. These use cases allow monitoring users to view the dashboard, asset monitor, node monitor, current asset status, and live updates.

The Monitor App is designed to provide operational visibility. It helps users observe where assets are located, whether nodes are online, whether assets are in place or in transit, and whether any warnings or alerts require attention.

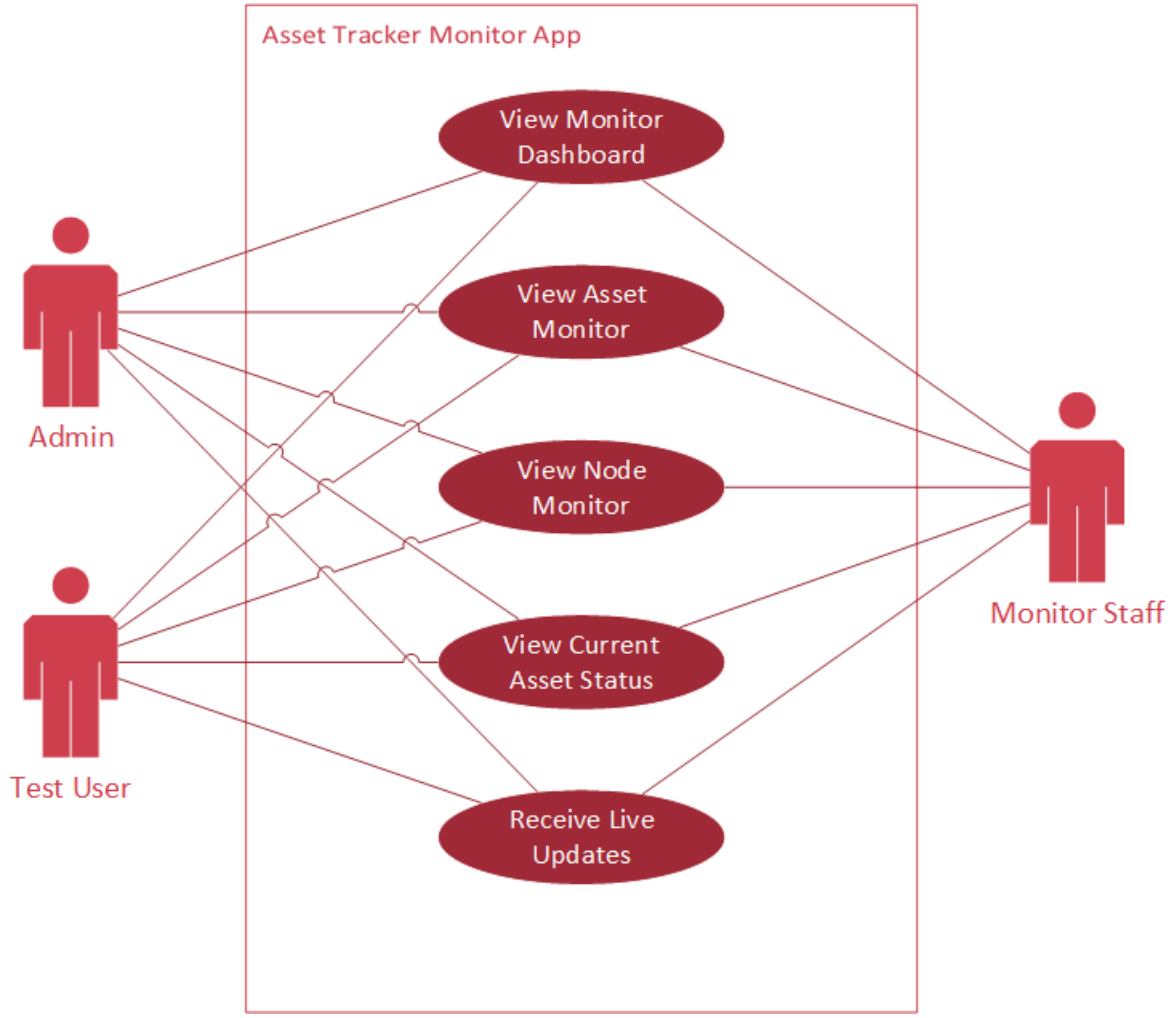

Figure 8. 8 Monitor app access use case diagram.

8.8.1. UC-026 – View Monitor Dashboard

Table 8. 8. 1 Data of use case UC-026.

Use Case ID:	UC-026
Name:	View Monitor Dashboard
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff

Description:	Allows monitoring-side users to view the main Monitor App dashboard.
Pre-Condition:	The user is logged in to the Monitor App with monitor permission.
Basic Flows:	1. The user opens the Monitor App dashboard. 2. The frontend requests asset, node, and alert summary data. 3. The backend returns current monitoring data. 4. The dashboard calculates status counts and warning counts. 5. The user reviews the monitoring overview.
Alternative Flows:	1. If data loading fails, the dashboard displays an error or fallback state. 2. If no warnings exist, warning cards show zero values.
Post-Condition:	The user can see the current monitoring overview.

8.8.2. UC-027 – View Asset Monitor

Table 8. 8. 2 Data of use case UC-027.

Use Case ID:	UC-027
Name:	View Asset Monitor
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows monitoring users to view asset status, current location, assigned room, and movement state.
Pre-Condition:	The user has Monitor App access.
Basic Flows:	1. The user opens the Assets page in the Monitor App. 2. The frontend requests monitored asset data. 3. The backend returns active assets and flow status values. 4. The frontend displays the current room, assigned room, status, and action buttons. 5. The user searches, sorts, or reviews the asset list.
Alternative Flows:	1. If no assets exist, the table shows an empty state. 2. If the asset has a warning state, the row or badge highlights it.
Post-Condition:	The user can monitor asset locations and movement state.

8.8.3. UC-028 – View Node Monitor

Table 8. 8. 3 Data of use case UC-028.

Use Case ID:	UC-028
Name:	View Node Monitor
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows monitoring users to view node status and availability.
Pre-Condition:	The user has Monitor App access.
Basic Flows:	1. The user opens the Nodes page in the Monitor App. 2. The frontend requests node monitoring data. 3. The backend returns node records and status values. 4. The frontend displays online, offline, disabled, and unknown statuses. 5. The user reviews node availability.
Alternative Flows:	1. If nodes are offline, the UI highlights the offline status. 2. If no nodes are registered, the table shows an empty state.
Post-Condition:	The user can identify whether sensor nodes are available for tracking.

8.8.4. UC-029 – View Current Asset Status

Table 8. 8. 4 Data of use case UC-029.

Use Case ID:	UC-029
Name:	View Current Asset Status
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows monitoring users to check whether an asset is in place, in transit, pending approval, or in the wrong location.
Pre-Condition:	The user has Monitor App access, and asset data exists.
Basic Flows:	1. The user opens the asset monitor table. 2. The frontend displays each asset flow status. 3. The system compares the current location, assigned room, and movement request state. 4. The frontend shows a status badge for each asset. 5. The user identifies assets that need attention.
Alternative Flows:	1. If an asset is not detected at its assigned room, the status may show pending placement or wrong location. 2. If a movement is approved, the status may show in transit.
Post-Condition:	The user understands the current operational status of each monitored asset.

8.8.5. UC-030 – Receive Live Updates

Table 8. 8. 5 Data of use case UC-030.

Use Case ID:	UC-030
Name:	Receive Live Updates
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows frontend pages to update automatically when the backend broadcasts WebSocket events.
Pre-Condition:	The user is logged in, and the WebSocket connection is active.
Basic Flows:	1. The frontend opens a WebSocket connection after login. 2. The backend accepts the connection and keeps it active. 3. A backend event occurs, such as an asset update, node heartbeat, or movement change. 4. The backend broadcasts the event through WebSocket. 5. The frontend receives the event and updates local UI data without manual refresh.
Alternative Flows:	1. If the WebSocket disconnects, the frontend attempts to reconnect. 2. If an event payload is unsupported, the frontend ignores it safely.
Post-Condition:	The frontend displays current data based on live backend events.

8.9. Asset Alert and Activity

This section describes use cases related to alerts and activity logs. Alerts help users identify important problems such as unknown RFID tags, offline nodes, and unauthorized asset movement. Activity logs help users review important asset and node events that occurred in the system.

These use cases improve operational awareness by allowing users to detect tracking problems, investigate asset movement, and review the history of system events.

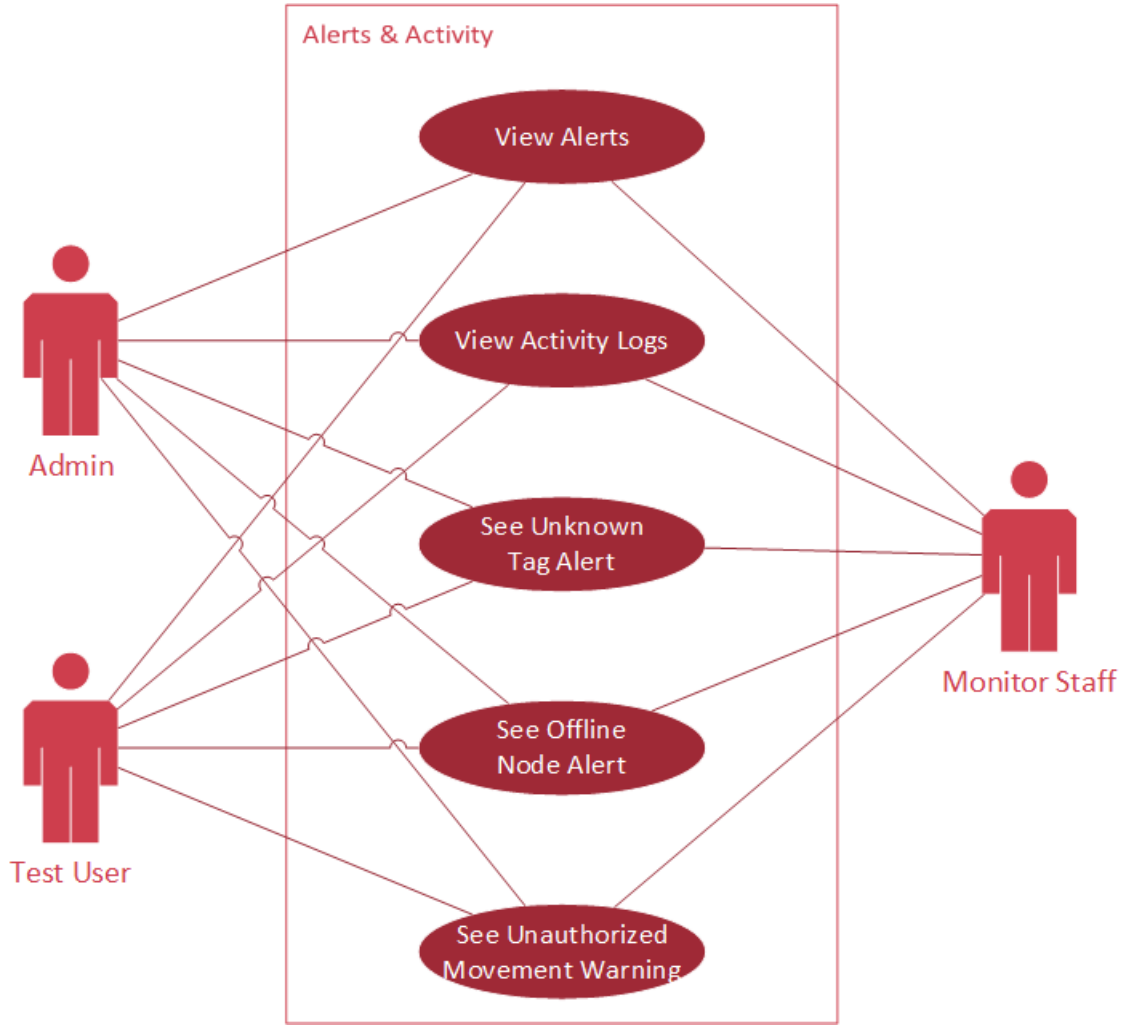

Figure 8. 9 Monitor app alert notification use case diagram.

8.9.1. UC-031 – View Alerts

Table 8. 9. 1 Data of use case UC-031.

Use Case ID:	UC-031
Name:	View Alerts
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows monitoring users to view system warnings and asset or node alerts.

Pre-Condition:	The user has Monitor App access.
Basic Flows:	1. The user opens the Alerts page. 2. The frontend requests current alerts. 3. The backend returns alert data or calculated warning states. 4. The frontend displays the alert category, severity, message, and time. 5. The user reviews alerts that require attention.
Alternative Flows:	1. If no alerts exist, the page shows an empty state. 2. If alert loading fails, the page shows an error.
Post-Condition:	The user can see current warnings and issues in the system.

8.9.2. UC-032 – View Activity Logs

Table 8. 9. 2 Data of use case UC-032.

Use Case ID:	UC-032
Name:	View Activity Logs
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Allows monitoring users to view system activity history.
Pre-Condition:	The user has Monitor App access.
Basic Flows:	1. The user opens the Activity page. 2. The frontend requests activity logs. 3. The backend returns recent activity records. 4. The frontend displays activity type, related asset or node, message, and timestamp. 5. The user reviews the operational history.
Alternative Flows:	1. If heartbeat events are hidden, the page only shows important activity types. 2. If no activity exists, the page shows an empty state.
Post-Condition:	The user can review past system events.

8.9.3. UC-033 – See Unknown Tag Alert

Table 8. 9. 3 Data of use case UC-033.

Use Case ID:	UC-033
Name:	See Unknown Tag Alert
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Shows an alert when an RFID tag is detected but not registered as an active asset.
Pre-Condition:	A checkpoint node detects an RFID tag and sends the detection to the backend.
Basic Flows:	1. The checkpoint node detects a tag. 2. The backend searches for an active asset with the tag ID. 3. No matching active asset is found. 4. The backend creates or broadcasts an unknown tag alert. 5. The Monitor App displays the warning to the user.
Alternative Flows:	1. If the tag belongs to a deregistered asset, the system may show a deregistered/unknown warning. 2. If alert broadcasting fails, the issue may appear after a refresh.
Post-Condition:	The unknown tag detection is visible as a monitoring warning.

8.9.4. UC-034 – See Offline Node Alert

Table 8. 9. 4 Data of use case UC-034.

Use Case ID:	UC-034
Name:	See Offline Node Alert
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Shows an alert when a node is considered offline due to missing heartbeat updates.
Pre-Condition:	A node has been assigned and is expected to send heartbeat updates.
Basic Flows:	1. The backend offline monitor checks node heartbeat timestamps. 2. The backend finds a node with an expired last ping time. 3. The backend sets the node status to offline. 4. The backend broadcasts the node offline update. 5. The Monitor App displays an offline node alert.
Alternative Flows:	1. If the node is disabled, it may be excluded from offline warning logic. 2. If the node sends a new heartbeat, the system changes it back to online.
Post-Condition:	The offline node appears unavailable and may trigger a warning.

8.9.5. UC-035 – See Unauthorized Movement Warning

Table 8. 9. 5 Data of use case UC-035.

Use Case ID:	UC-035
Name:	See Unauthorized Movement Warning
Actor:	❖ Admin ❖ Test User ❖ Monitor Staff
Description:	Shows a warning when an asset is detected in a room without an approved movement request.
Pre-Condition:	The asset is active and is detected by a checkpoint node outside its expected room.
Basic Flows:	1. A checkpoint node detects an asset RFID tag. 2. The backend loads the asset and current assignment. 3. The backend checks whether there is an approved movement request. 4. No valid approved movement matches the detected location. 5. The backend marks or reports the asset as unauthorized movement or wrong location. 6. The Monitor App displays a warning.
Alternative Flows:	1. If a matching approved movement exists, the system may complete the movement instead. 2. If the asset is pending placement, the system may treat the first correct detection as placement.
Post-Condition:	The asset warning is visible to enable staff to investigate the asset's movement.

8.10. Node Real-Time Communication

This section describes the real-time communication between sensor nodes, the MQTT broker, the backend system, and frontend applications. Sensor nodes send registration scans, RFID detections, and heartbeat messages through MQTT. The MQTT broker delivers these messages to the backend system.

After receiving a valid node message, the backend updates the corresponding asset or node data and broadcasts the relevant changes via WebSocket. This real-time communication allows the Registration App and Monitor App to update automatically without manual refresh.

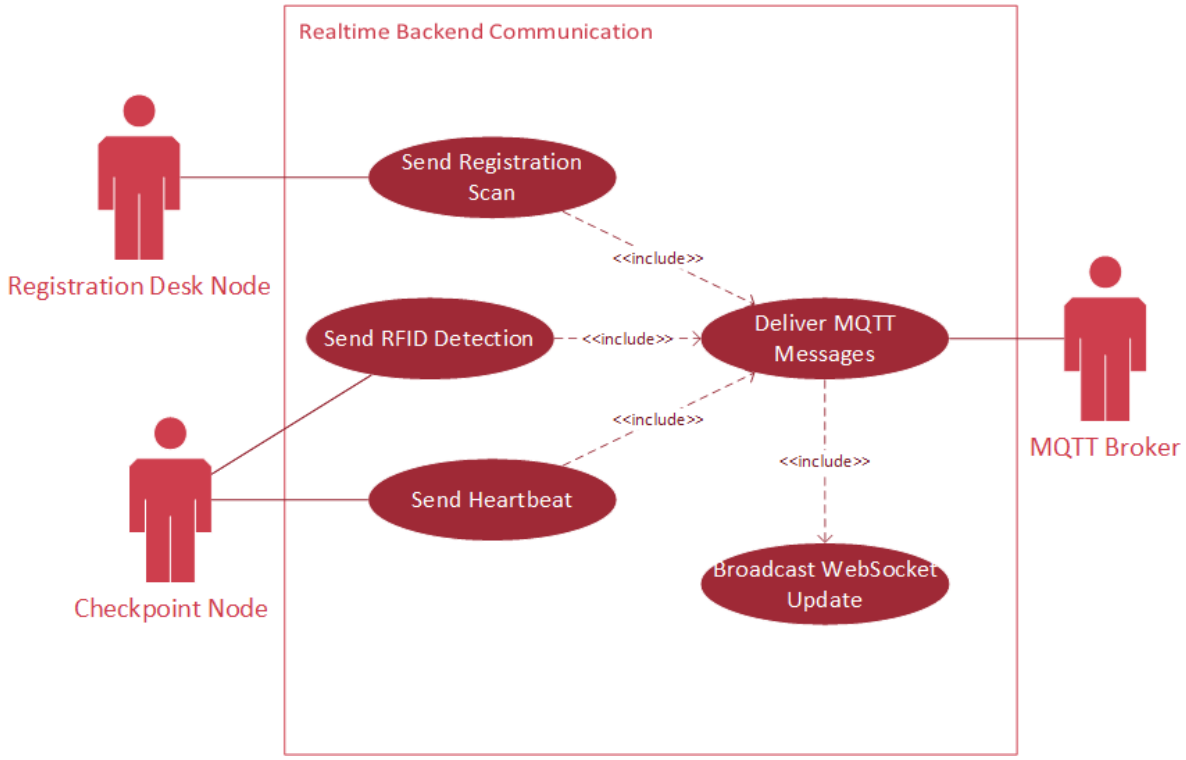

Figure 8. 10 System backend communication use case diagram.

8.10.1. UC-036 – Send Registration Scan

Table 8. 10. 1 Data of use case UC-036.

Use Case ID:	UC-036
Name:	Send Registration Scan
Actor:	❖ Registration Desk Node
Description:	Allows the registration desk node to send RFID scan data to the backend through MQTT.
Pre-Condition:	The registration desk node is powered, configured, connected to Wi-Fi, and connected to MQTT.
Basic Flows:	1. The node reads an RFID tag at the registration desk. 2. The node builds a registration scan payload. 3. The node publishes the payload to the MQTT broker. 4. The MQTT broker delivers the payload to the backend. 5. The backend processes the scan and broadcasts it to the Registration App.
Alternative Flows:	1. If MQTT is disconnected, the node may retry after reconnection. 2. If the payload is invalid, the backend rejects it.
Post-Condition:	The scanned tag data is available to the registration workflow.

8.10.2. UC-037 – Send RFID Detection

Table 8. 10. 2 Data of use case UC-037.

Use Case ID:	UC-037
Name:	Send RFID Detection
Actor:	❖ Checkpoint Node
Description:	Allows a checkpoint node to report RFID-tagged assets detected in its room or area.
Pre-Condition:	The checkpoint node is assigned, online, and connected to the RFID reader and MQTT broker.
Basic Flows:	1. The checkpoint node detects an RFID tag. 2. The firmware reads the tag ID. 3. The node builds a detection payload with device ID and tag ID. 4. The node publishes the detection to MQTT. 5. The backend receives and processes the detection. 6. The backend updates asset location or creates a warning.
Alternative Flows:	1. If the tag is unknown, the backend creates an unknown tag warning. 2. If the node is disabled or unknown, the backend rejects or warns about the detection.
Post-Condition:	The backend has processed the asset detection event.

8.10.3. UC-038 – Send Heartbeat

Table 8. 10. 3 Data of use case UC-038.

Use Case ID:	UC-038
Name:	Send Heartbeat
Actor:	❖ Checkpoint Node
Description:	Allows a checkpoint node to inform the backend that it is online periodically.
Pre-Condition:	The node is powered and connected to MQTT.
Basic Flows:	1. The firmware reaches the heartbeat interval. 2. The node creates a heartbeat payload with device ID and status. 3. The node publishes the heartbeat to MQTT. 4. The backend receives the heartbeat. 5. The backend updates last_ping_at and node status to online. 6. The backend broadcasts the node heartbeat update.
Alternative Flows:	1. If MQTT is disconnected, the heartbeat is not delivered until reconnection. 2. If the node is unknown, the backend may create or update the discovery state.
Post-Condition:	The backend records the node as recently online.

8.10.4. UC-039 – Deliver MQTT Messages

Table 8. 10. 4 Data of use case UC-039.

Use Case ID:	UC-039
Name:	Deliver MQTT Messages
Actor:	❖ MQTT Broker
Description:	Delivers scan, detection, heartbeat, and command messages between sensor nodes and the backend.
Pre-Condition:	Nodes and the backend are connected to the MQTT broker and subscribed to the correct topics.
Basic Flows:	1. A node or backend publishes an MQTT message. 2. The MQTT broker receives the message on a topic.

	3. The broker matches the topic with subscribed clients. 4. The broker delivers the message to the backend or node. 5. The receiver processes the message according to the topic type.
Alternative Flows:	1. If the topic is wrong, the intended receiver does not receive the message. 2. If the client is disconnected, delivery may fail depending on the MQTT session settings.
Post-Condition:	The MQTT broker delivers the published message to every connected client whose subscription matches the topic.

8.10.5. UC-040 – Broadcast WebSocket Update

Table 8. 10. 5 Data of use case UC-040.

Use Case ID:	UC-040
Name:	Broadcast WebSocket Update
Actor:	❖ Backend System
Description:	Sends live update events from the backend to connected Registration App and Monitor App clients.
Pre-Condition:	At least one frontend client is connected through WebSocket.
Basic Flows:	1. A backend service completes a relevant update. 2. The backend creates a WebSocket event payload. 3. The WebSocket manager sends the event to connected clients. 4. The frontend receives the event. 5. The frontend updates the affected table, badge, alert, or dashboard count.
Alternative Flows:	1. If a client is disconnected, it will not receive the live event and may need to refresh or reconnect. 2. If the event type is unknown to the frontend, it is ignored safely.
Post-Condition:	Connected frontend clients show updated data without manual refresh.

8.11. System Settings

This section describes the use case for managing system settings. The System Settings page allows authorized users to view and manage configuration values related to the system or MQTT communication.

Because system and MQTT settings can affect how the backend communicates with sensor nodes, this feature should only be available to authorized users. The system validates submitted values before saving changes and prevents unauthorized roles from modifying settings.

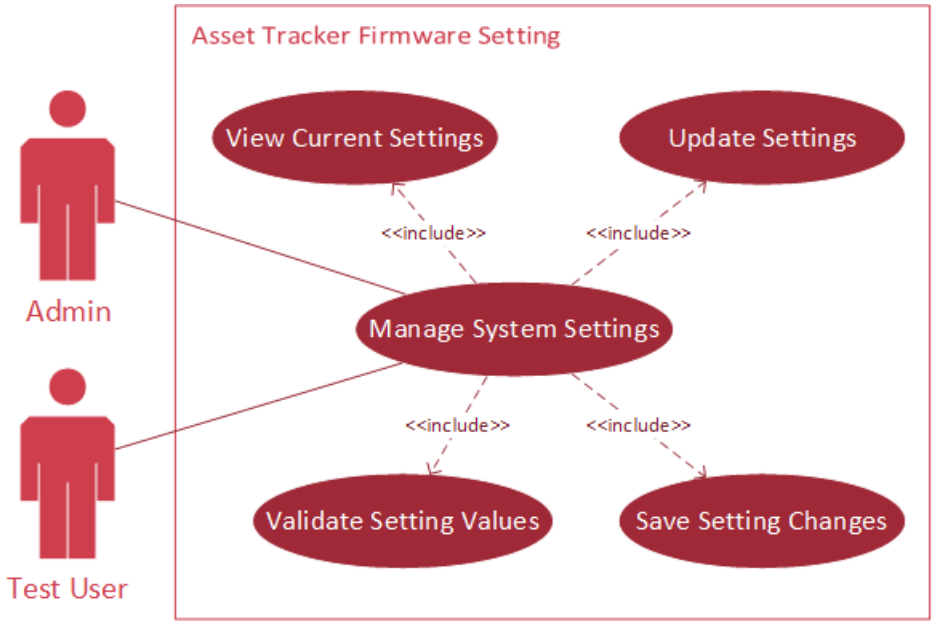

Figure 8. 11 System settings management use case diagram.

8.11.1. UC-041 – Manage System Settings

Table 8. 11. 1 Data of use case UC-041.

Use Case ID:	UC-041
Name:	Manage System Settings
Actor:	❖ Admin ❖ Test User
Description:	Allows authorized users to view and manage system configuration settings, including MQTT-related settings used for communication between the sensor nodes and the backend system.
Pre-Condition:	The user is logged in with a role that allows access to the System Settings page. The backend settings service is available.
Basic Flows:	1. The user opens the System Settings page. 2. The frontend checks whether the user role is allowed to access system settings. 3. The frontend requests the current system or MQTT settings from the backend. 4. The backend validates the user's permission and returns the current settings. 5. The page displays the current configuration values to the user. 6. The user updates one or more setting values. 7. The frontend sends the updated settings to the backend. 8. The backend validates the submitted values. 9. The backend saves the valid setting changes. 10. The frontend displays a success message and refreshes the displayed settings.
Alternative Flows:	1. If the user role is not allowed, the system blocks access or shows an access denied message. 2. If the backend cannot load the current settings, the page shows an error message. 3. If the submitted setting values are invalid, the backend rejects the update, and the frontend shows a validation error. 4. If the user cancels or leaves the page before saving, no setting changes are applied.
Post-Condition:	The system settings are displayed to the user. If valid changes were submitted, the updated settings are saved and used by the system in accordance with the configuration rules.

9. Requirements-to-Use-Case Matrix

This section provides traceability between requirements and use cases. The traceability matrix shows how one or more use cases support each user requirement, functional requirement, and non-functional requirement.

The purpose of the matrix is to confirm that all requirements are covered by the system behavior described in the use cases. It also helps identify whether any requirement is missing a related workflow or whether any use case does not support a clear requirement.

9.1. User Requirements to Use Case

This subsection maps each user requirement to the use cases that support it. The mapping shows how specific system interactions represent the needs of users and operational stakeholders.

This table helps verify that every user requirement has at least one related use case. It also helps readers understand which use cases satisfy each user requirement.

Table 9. 1 User requirement-to-use case matrix table.

No.	Requirement ID	Use Case ID	Explanation
1.	UR-001	UC-002 – Login UC-003 – Logout UC-006 – View Registration Dashboard UC-026 – View Monitor Dashboard	These use cases control user access to the system and redirect users to allowed pages after login.
2.	UR-002	UC-004 – Change Own Password UC-007 – Open Change Password Page	These use cases allow authenticated users to open the account security page and update their own password.
3.	UR-003	UC-005 – Manage User Accounts UC-008 – Open User Accounts Page	These use cases allow the admin to access and manage user accounts.
4.	UR-004	UC-006 – View Registration Dashboard	This use case displays dashboard cards and links based on the logged-in user's role.
5.	UR-005	UC-009 – Open Node Registration Page	This use case allows authorized users to access the Node Registration page.
6.	UR-006	UC-011 – View Nodes UC-028 – View Node Monitor	These use cases allow users to view node records and node availability.
7.	UR-007	UC-012 – Assign Node UC-013 – Edit Node Assignment UC-014 – Identify Node by Blink UC-015 – Unassign Node UC-016 – Delete Node	These use cases cover the main node management actions.
8.	UR-008	UC-010 – Open Asset Registration Page UC-017 – View Assets	These use cases allow authorized users to access and view registration-side asset data.
9.	UR-009	UC-018 – Register Asset UC-020 – Scan RFID Tag at Registration Desk UC-036 – Send Registration Scan	These use cases support registering a new RFID-tagged asset through the registration desk node.
10.	UR-010	UC-019 – Deregister Asset	This use case allows authorized users to deactivate or deregister an active asset.
11.	UR-011	UC-021 – Request Asset Movement UC-022 – Cancel Movement Request	These use cases allow monitor-side users to request or cancel asset movement.
12.	UR-012	UC-023 – Approve Movement Request UC-024 – Reject Movement Request	These use cases allow registration-side users to make movement decisions.
13.	UR-013	UC-025 – Complete Approved Movement UC-037 – Send RFID Detection	These use cases complete an approved movement after the asset is detected at the destination.
14.	UR-014	UC-026 – View Monitor Dashboard UC-027 – View Asset Monitor UC-028 – View Node Monitor UC-029 – View Current Asset Status UC-031 – View Alerts UC-032 – View Activity Logs	These use cases allow monitor users to observe asset, node, alert, and activity information.
15.	UR-015	UC-033 – See Unknown Tag Alert UC-034 – See Offline Node Alert UC-035 – See Unauthorized Movement Warning	These use cases show the warning conditions that help staff identify tracking problems.

16.	UR-016	UC-036 – Send Registration Scan UC-037 – Send RFID Detection UC-038 – Send Heartbeat UC-039 – Deliver MQTT Messages	These use cases describe node-to-backend communication through MQTT.
17.	UR-017	UC-030 – Receive Live Updates UC-040 – Broadcast WebSocket Update	These use cases support automatic frontend updates after backend changes.
18.	UR-018	UC-002 – Login UC-005 – Manage User Accounts UC-006 – View Registration Dashboard UC-008 – Open User Accounts Page UC-009 – Open Node Registration Page UC-010 – Open Asset Registration Page UC-026 – View Monitor Dashboard	These use cases enforce role-based access across login, dashboard, user management, node management, asset management, and monitoring pages.
19.	UR-019	UC-041 – Manage System Settings	This use case allows authorized users to view and manage system or MQTT configuration settings.
20.	UR-020	UC-001 – Create First Admin Account	This use case allows the system to be initialized when no admin account exists.

9.2. Functional Requirements to Use Case and User Requirements

This subsection maps functional requirements to their related user requirements and use cases. Functional requirements describe the detailed system behavior needed to support user goals, while use cases describe how users or system actors interact with those functions.

This table helps verify that the functional requirements are not isolated technical statements. Each functional requirement group is connected to a user need and to one or more use cases that describe the related workflow.

Table 9. 2 Functional requirements-to-user requirements matrix table.

No.	Functional Requirement(s)	User Requirement(s)	Use Case(s)	Relation
1.	FR-001 FR-002 FR-003 FR-004	UR-001 UR-018	❖ UC-002 – Login ❖ UC-003 – Logout	These functional requirements support user authentication, session creation, role identification, and secure logout.
2.	FR-005 FR-006	UR-002	❖ UC-004 – Change Own Password ❖ UC-007 – Open Change Password Page	These requirements enable authenticated users to open the password page and securely change their own password.
3.	FR-007 FR-008	UR-003 UR-018	❖ UC-005 – Manage User Accounts ❖ UC-008 – Open User Accounts Page	These requirements support admin user management and protect against unsafe admin account actions.
4.	FR-009 FR-062 FR-063	UR-001 UR-018	❖ UC-002 – Login ❖ UC-005 – Manage User Accounts ❖ UC-006 – View Registration Dashboard ❖ UC-008 – Open User Accounts Page ❖ UC-009 – Open Node Registration Page ❖ UC-010 – Open Asset Registration Page ❖ UC-026 – View Monitor Dashboard ❖ UC-041 – Manage System Settings	These requirements enforce role-based access in both frontend navigation and backend API requests.
5.	FR-010 FR-011 FR-012	UR-004 UR-018	❖ UC-006 – View Registration Dashboard	These requirements support the role-based Registration Dashboard and hide unavailable dashboard information.
6.	FR-013	UR-005 UR-018	❖ UC-009 – Open Node Registration Page	This requirement allows authorized users to access the Node Registration page.
7.	FR-014 FR-015	UR-006 UR-014	❖ UC-011 – View Nodes ❖ UC-028 – View Node Monitor	These requirements support displaying node records, node details, and node availability.

8.	FR-016 FR-017 FR-018 FR-019 FR-020 FR-021	UR-007 UR-016	❖ UC-012 – Assign Node ❖ UC-013 – Edit Node Assignment ❖ UC-014 – Identify Node by Blink ❖ UC-015 – Unassign Node ❖ UC-016 – Delete Node ❖ UC-039 – Deliver MQTT Messages	These requirements support node assignment, editing, identification, unassignment, deletion, and MQTT-based blink commands.
9.	FR-022 FR-023	UR-008 UR-014	❖ UC-010 – Open Asset Registration Page ❖ UC-017 – View Assets ❖ UC-027 – View Asset Monitor ❖ UC-029 – View Current Asset Status	These requirements support opening asset-related pages and displaying asset information.
10.	FR-024 FR-025 FR-026 FR-027 FR-028	UR-009 UR-016	❖ UC-018 – Register Asset ❖ UC-020 – Scan RFID Tag at Registration Desk ❖ UC-036 – Send Registration Scan	These requirements support RFID-based asset registration through the registration desk node.
11.	FR-029 FR-030	UR-010	❖ UC-019 – Deregister Asset	These requirements support deregistering an asset so it is no longer treated as actively tracked.
12.	FR-031 FR-032 FR-033 FR-034	UR-011	❖ UC-021 – Request Asset Movement ❖ UC-022 – Cancel Movement Request	These requirements support requesting asset movement, preventing duplicate requests, and canceling pending requests.
13.	FR-035 FR-036 FR-037 FR-038 FR-039	UR-012	❖ UC-023 – Approve Movement Request ❖ UC-024 – Reject Movement Request	These requirements support registration-side approval and rejection of movement requests.
14.	FR-040 FR-041	UR-013 UR-016	❖ UC-025 – Complete Approved Movement ❖ UC-037 – Send RFID Detection	These requirements support completing an approved movement when the asset is detected at the approved destination.
15.	FR-042 FR-043 FR-044 FR-045 FR-046	UR-014	❖ UC-026 – View Monitor Dashboard ❖ UC-027 – View Asset Monitor ❖ UC-028 – View Node Monitor ❖ UC-029 – View Current Asset Status ❖ UC-031 – View Alerts ❖ UC-032 – View Activity Logs	These requirements support monitoring asset status, node status, alerts, and activity logs.
16.	FR-047 FR-048 FR-049 FR-050 FR-051	UR-015	❖ UC-031 – View Alerts ❖ UC-033 – See Unknown Tag Alert ❖ UC-034 – See Offline Node Alert ❖ UC-035 – See Unauthorized Movement Warning ❖ UC-037 – Send RFID Detection ❖ UC-038 – Send Heartbeat	These requirements support detecting warnings and displaying alerts for unknown tags, offline nodes, and unauthorized asset movement.
17.	FR-052 FR-053 FR-054 FR-055 FR-056	UR-016	❖ UC-036 – Send Registration Scan ❖ UC-037 – Send RFID Detection ❖ UC-038 – Send Heartbeat ❖ UC-039 – Deliver MQTT Messages	These requirements support MQTT communication between sensor nodes, the MQTT broker, and the backend system.
18.	FR-057 FR-058 FR-059 FR-060 FR-061	UR-017	❖ UC-030 – Receive Live Updates ❖ UC-040 – Broadcast WebSocket Update	These requirements support WebSocket connections, live frontend updates, reconnection behavior, and the safe handling of unknown event types.
19.	FR-064 FR-065 FR-066 FR-067 FR-068	UR-019 UR-018	❖ UC-041 – Manage System Settings	These requirements support opening, retrieving, updating, validating, and protecting system or MQTT settings.
20.	FR-069	UR-020	❖ UC-001 – Create First Admin Account	This requirement supports creating the first admin account when none exists.

9.3. Non-Functional Requirements to Use Case and User Requirements

This subsection maps non-functional requirements to related user requirements and use cases. Non-functional requirements describe quality attributes such as security, reliability, availability, performance, usability, maintainability, compatibility, data integrity, auditability, and scalability.

Some non-functional requirements apply to many use cases because they affect the system's overall behavior and quality. This matrix illustrates how each quality attribute supports the user experience, system operation, or technical reliability of the Hospital Asset Tracker System.

Table 9. 3 Non-functional requirements-to-use case and user requirements matrix table.

No.	Non-Functional Requirement(s)	User Requirement(s)	Use Case(s)	Relation
1.	NFR-001 NFR-002 NFR-004 NFR-006	UR-001 UR-018	❖ UC-002 – Login ❖ UC-003 – Logout ❖ UC-006 – View Registration Dashboard ❖ UC-009 – Open Node Registration Page ❖ UC-010 – Open Asset Registration Page ❖ UC-026 – View Monitor Dashboard ❖ UC-041 – Manage System Settings	These NFRs support secure access, role restriction, disabled-account rejection, and session clearing.
2.	NFR-003	UR-002 UR-020	❖ UC-001 – Create First Admin Account ❖ UC-004 – Change Own Password	This NFR supports secure password storage for first-admin creation and normal password changes.
3.	NFR-005	UR-003 UR-018	❖ UC-005 – Manage User Accounts	This NFR supports admin account safety rules, such as preventing the last active admin from being disabled or demoted.
4.	NFR-007 NFR-008	UR-001 UR-016 UR-019	❖ UC-002 – Login ❖ UC-039 – Deliver MQTT Messages ❖ UC-041 – Manage System Settings	These NFRs support secure deployment and protection of sensitive configuration values.
5.	NFR-009 NFR-010 NFR-013	UR-014 UR-015 UR-017	❖ UC-030 – Receive Live Updates ❖ UC-031 – View Alerts ❖ UC-033 – See Unknown Tag Alert ❖ UC-034 – See Offline Node Alert ❖ UC-035 – See Unauthorized Movement Warning ❖ UC-040 – Broadcast WebSocket Update	These NFRs support application reliability when WebSocket updates, unknown events, or warning conditions occur.
6.	NFR-011 NFR-012	UR-015 UR-016	❖ UC-034 – See Offline Node Alert ❖ UC-036 – Send Registration Scan ❖ UC-037 – Send RFID Detection ❖ UC-038 – Send Heartbeat ❖ UC-039 – Deliver MQTT Messages	These NFRs support reliable node communication, MQTT validation, and offline-node detection.
7.	NFR-014 NFR-015	UR-001 to UR-020	All main user-facing use cases: ❖ UC-001 to UC-019 ❖ UC-021 to UC-035 ❖ UC-041	These NFRs support backend availability and clear frontend error messages when backend, database, MQTT, or WebSocket services are unavailable.
8.	NFR-016 NFR-017 NFR-018	UR-004 UR-014 UR-017	❖ UC-006 – View Registration Dashboard ❖ UC-011 – View Nodes ❖ UC-017 – View Assets ❖ UC-026 – View Monitor Dashboard ❖ UC-027 – View Asset Monitor ❖ UC-028 – View Node Monitor ❖ UC-030 – Receive Live Updates	These NFRs support reasonable loading time, timely frontend updates, and usable table searching, sorting, filtering, and pagination.
9.	NFR-019 NFR-020 NFR-021 NFR-022 NFR-023 NFR-024	UR-001 UR-004 UR-005 UR-008 UR-011 UR-012 UR-014 UR-018	❖ UC-006 – View Registration Dashboard ❖ UC-009 – Open Node Registration Page ❖ UC-010 – Open Asset Registration Page ❖ UC-017 – View Assets ❖ UC-021 – Request Asset Movement ❖ UC-023 – Approve Movement Request ❖ UC-026 – View Monitor Dashboard ❖ UC-031 – View Alerts	These NFRs support usability, responsive layout, clear status badges, clear feedback messages, and hiding actions that the user role is not allowed to perform.

10.	NFR-025 NFR-026 NFR-027 NFR-028	UR-001 to UR-020	❖ All use cases	These NFRs support maintainable code structure, reusable logic, consistent naming, and tests for critical workflows.
11.	NFR-029	UR-001 UR-004 UR-008 UR-014 UR-018 UR-019 UR-020	❖ UC-002 to UC-035 ❖ UC-041	This NFR supports browser compatibility for the Registration App, Monitor App, first-admin setup page, and system settings page.
12.	NFR-030	UR-016	❖ UC-036 – Send Registration Scan ❖ UC-037 – Send RFID Detection ❖ UC-038 – Send Heartbeat ❖ UC-039 – Deliver MQTT Messages	This NFR supports compatibility between the selected ESP32 hardware, RFID reader, Wi-Fi connection, and MQTT communication setup.
13.	NFR-031 NFR-032 NFR-033 NFR-034	UR-009 UR-010 UR-011 UR-012 UR-013 UR-015 UR-016	❖ UC-018 – Register Asset ❖ UC-019 – Deregister Asset ❖ UC-021 – Request Asset Movement ❖ UC-022 – Cancel Movement Request ❖ UC-023 – Approve Movement Request ❖ UC-024 – Reject Movement Request ❖ UC-025 – Complete Approved Movement ❖ UC-037 – Send RFID Detection	These NFRs support data integrity for asset registration, movement state, assigned room, current room, flow status, and RFID detection.
14.	NFR-035	UR-014 UR-015	❖ UC-031 – View Alerts ❖ UC-032 – View Activity Logs ❖ UC-033 – See Unknown Tag Alert ❖ UC-034 – See Offline Node Alert ❖ UC-035 – See Unauthorized Movement Warning	This NFR supports auditability through activity records, alerts, warnings, movement decisions, and node or asset events.
15.	NFR-036	UR-005 UR-006 UR-008 UR-014 UR-016	❖ UC-011 – View Nodes ❖ UC-012 – Assign Node ❖ UC-017 – View Assets ❖ UC-026 – View Monitor Dashboard ❖ UC-027 – View Asset Monitor ❖ UC-028 – View Node Monitor ❖ UC-036 – Send Registration Scan ❖ UC-037 – Send RFID Detection ❖ UC-038 – Send Heartbeat	This NFR supports future scalability when additional checkpoint nodes and tracked assets are added.